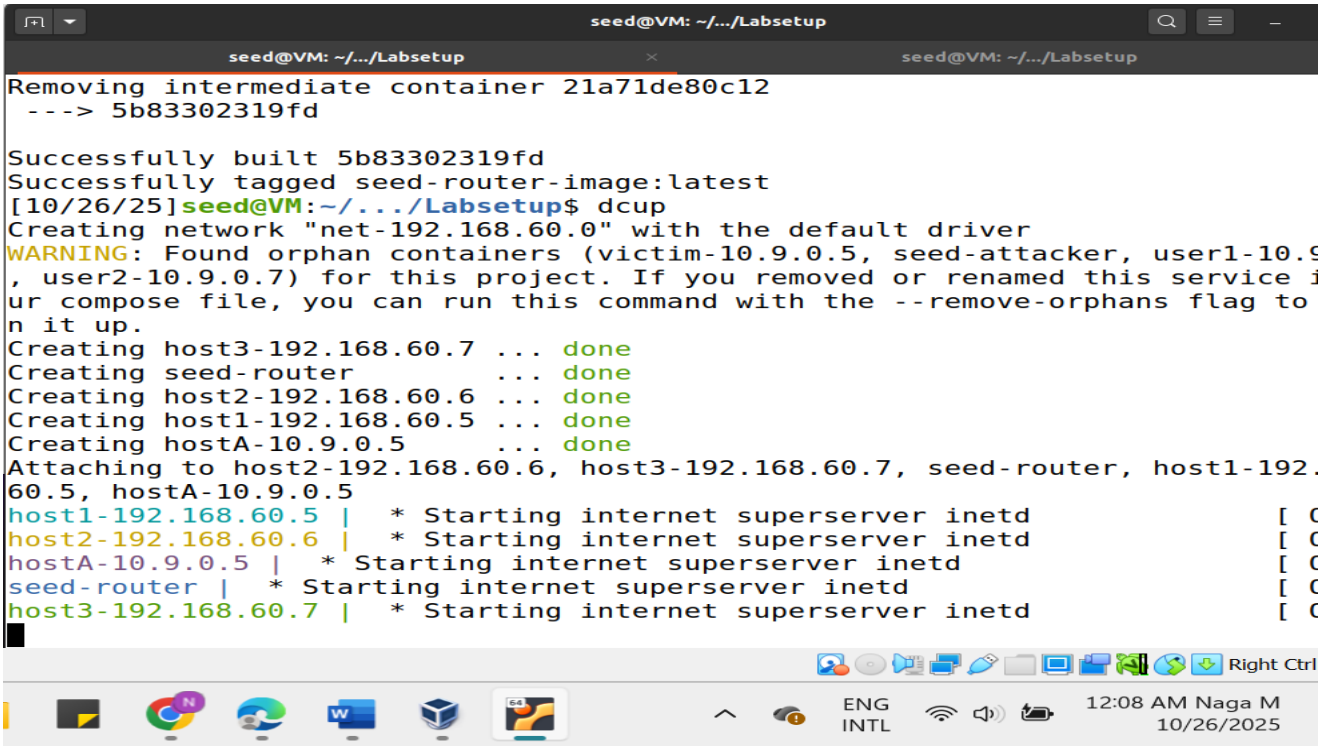
Firewall_Exploration

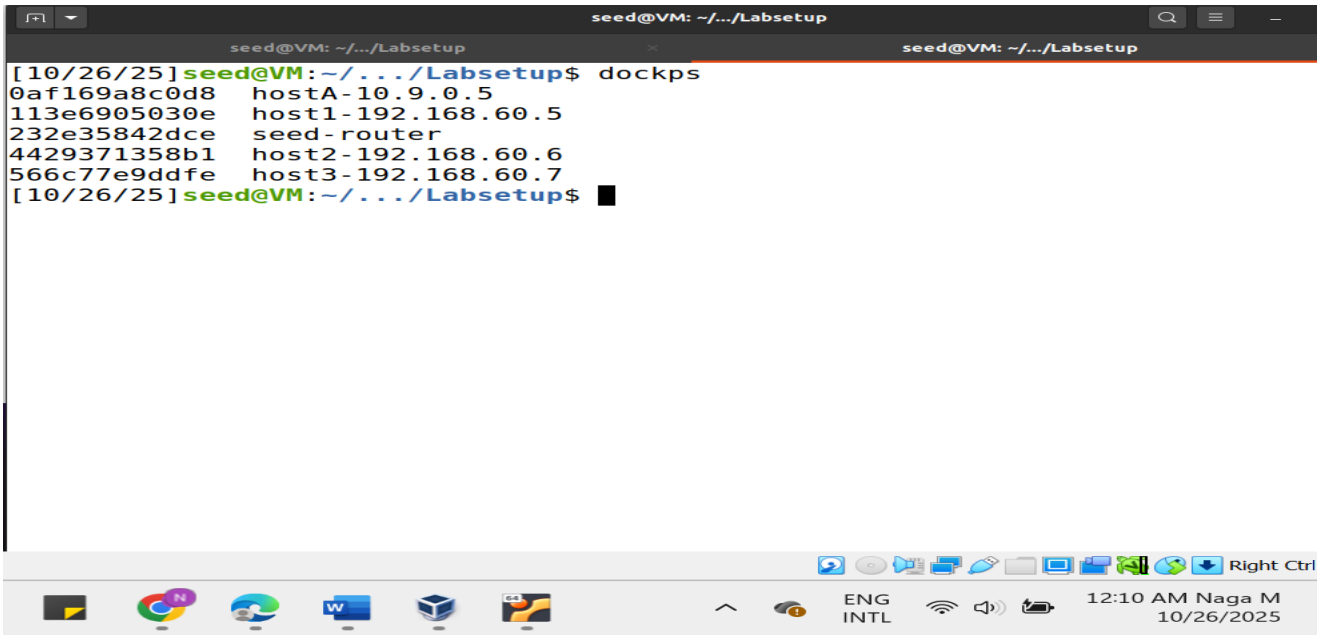
LAB Setup Environment

Run the SEED VM>> open Firefox in that>> go to the seed labs and then click on the seed labs >> network security > > Select the Firewall Exploration attack >>> Download the .zip file.

Copy this zip to Documents (any other folder in VM), here I moved it to documents >> FIrewall >> there unzip the labsetup.zip file. From here right click and open the Terminal then we can directly move to Terminal.
Now, We need to Set up Containers

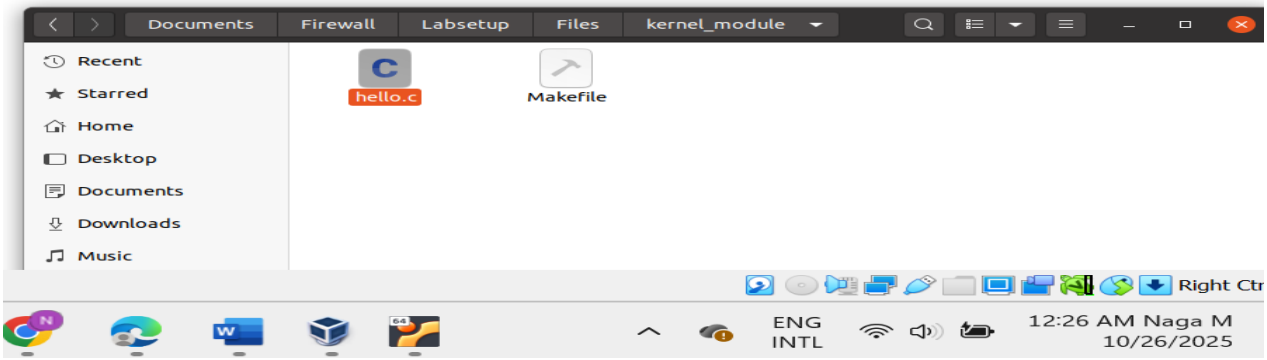


We first need to use the "dock ps" command to find out the ID of the container

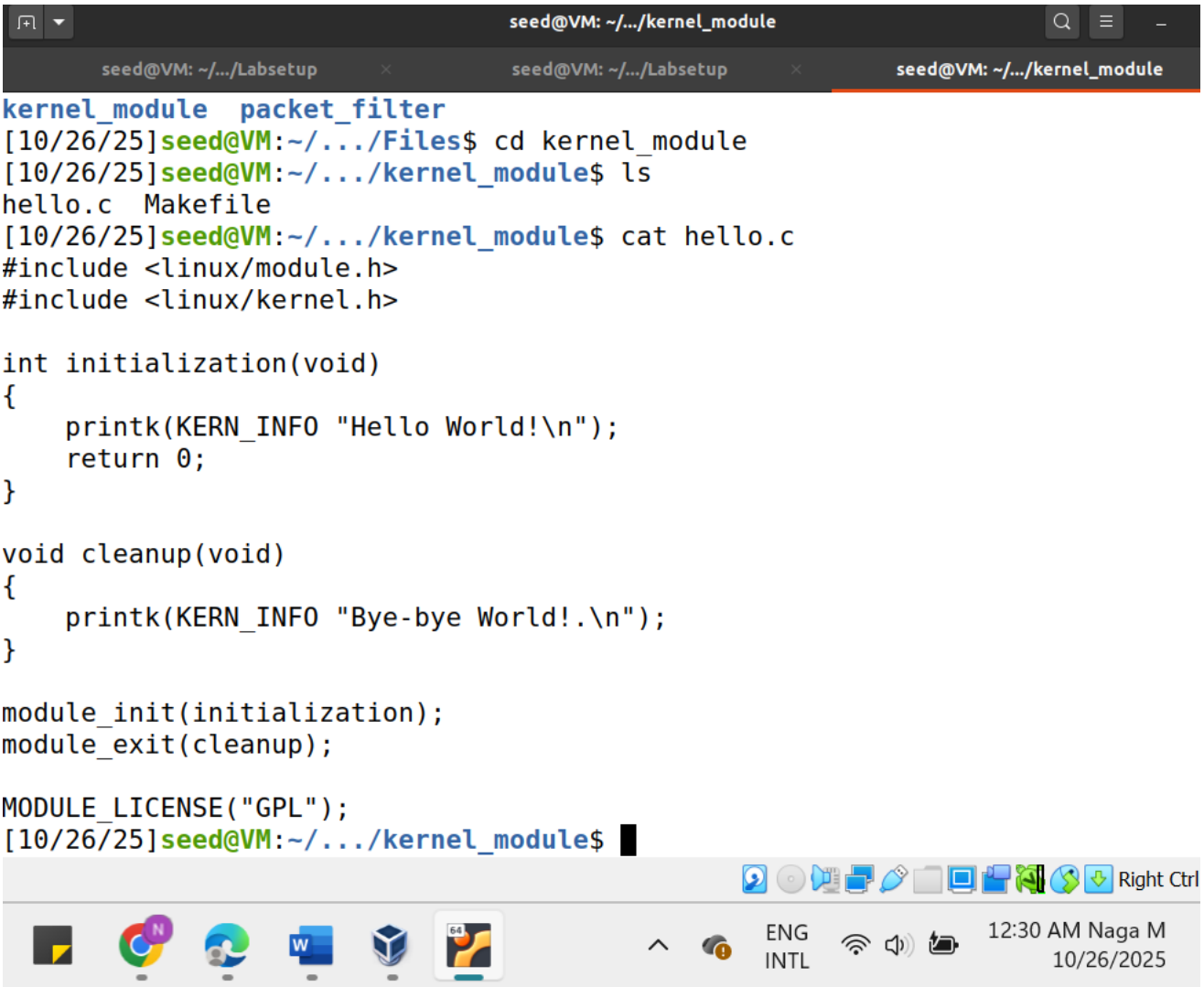


Task 1: Implementing a Simple Firewall

This task will drive us like how to build a firewall from scratch. In this task we can Understand how firewalls work at the kernel level. We had already "Hello World" kernel module code.



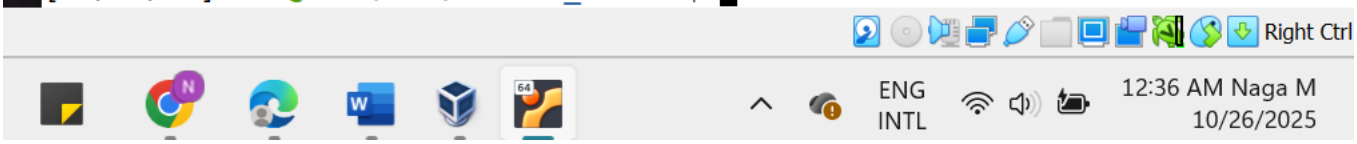
Find the hello.c file. It should be in the Labsetup/Files/packet_filter folder. Navigate there and in the terminal window type the command cat hello.c.



Here we can see the initialization function. Lets compile this module

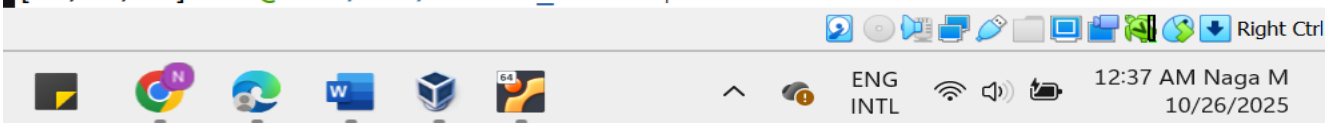
In this first we need to check the Makefile by using the command cat Makefile in the terminal. It should contain the commands to build the module for your current kernel

```
MODULE_LICENSE("GPL");
[10/26/25]seed@VM:~/.../kernel_module$ cat Makefile
obj-m += hello.o

all:
	make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules

clean:
	make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean

[10/26/25]seed@VM:~/.../kernel_module$
```



Compile the file and lets check hello.ko file created or not.

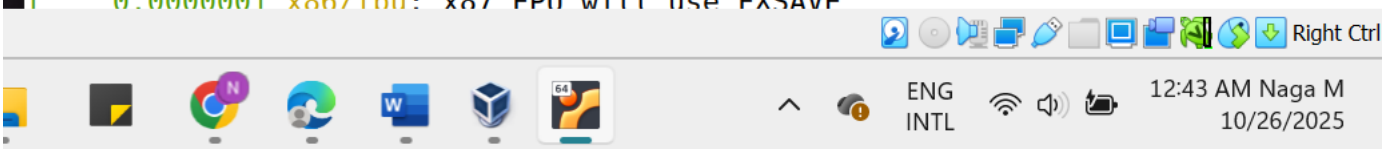
```
[10/26/25]seed@VM:~/.../kernel_module$ make
make -C /lib/modules/5.4.0-54-generic/build M=/home/seed/Documents/Firewall/
etup/Files/kernel_module modules
make[1]: Entering directory '/usr/src/linux-headers-5.4.0-54-generic'
  CC [M]  /home/seed/Documents/Firewall/Labsetup/Files/kernel_module/hello.c
  Building modules, stage 2.
  MODPOST 1 modules
  CC [M]  /home/seed/Documents/Firewall/Labsetup/Files/kernel_module/hello.m
  LD [M]  /home/seed/Documents/Firewall/Labsetup/Files/kernel_module/hello.k
make[1]: Leaving directory '/usr/src/linux-headers-5.4.0-54-generic'
[10/26/25]seed@VM:~/.../kernel_module$ ls -l hello.ko
-rw-rw-r-- 1 seed seed 3968 Oct 26 01:36 hello.ko
[10/26/25]seed@VM:~/.../kernel_module$
```



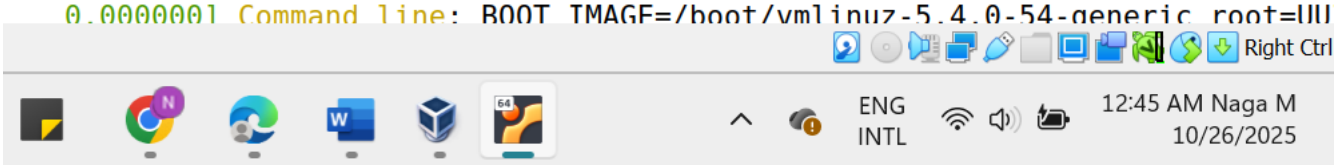
The generated kernel module is in hello.ko. You can use the following commands to load the module, list all modules, and remove the module. Also, you can use "modinfo hello.ko" to show information about a Linux Kernel module.

```
[10/26/25]seed@VM:~/.../kernel_module$ modinfo hello.ko
filename:       /home/seed/Documents/Firewall/Labsetup/Files/kernel_module/hello
.ko
license:        GPL
srcversion:     717A72281ACFAA8385B33A8
depends:
retpoline:      Y
name:           hello
vermagic:       5.4.0-54-generic SMP mod_unload
[10/26/25]seed@VM:~/.../kernel_module$
```

```
[10/26/25]seed@VM:~/.../kernel_module$ sudo insmod seedFilter.ko
insmod: ERROR: could not load module seedFilter.ko: No such file or directory
[10/26/25]seed@VM:~/.../kernel_module$ sudo insmod hello.ko
[10/26/25]seed@VM:~/.../kernel_module$ lsmod | grep hello
hello                  16384  0
[10/26/25]seed@VM:~/.../kernel_module$ sudo rmmod hello
[10/26/25]seed@VM:~/.../kernel_module$ dmesg
[    0.000000] Linux version 5.4.0-54-generic (buildd@lcy01-amd64-024) (gcc ve
ion 9.3.0 (Ubuntu 9.3.0-17ubuntu1~20.04)) #60-Ubuntu SMP Fri Nov 6 10:37:59 UT
2020 (Ubuntu 5.4.0-54.60-generic 5.4.65)
[    0.000000] Command line: BOOT_IMAGE=/boot/vmlinuz-5.4.0-54-generic root=UU
=a91f1a43-2770-4684-9fc3-b7abfd786c1d ro quiet splash
[    0.000000] KERNEL supported cpus:
[    0.000000]   Intel GenuineIntel
[    0.000000]   AMD AuthenticAMD
[    0.000000]   Hygon HygonGenuine
[    0.000000]   Centaur CentaurHauls
[    0.000000]   zhaoxin   Shanghai
[    0.000000] x86/fpu: x87 FPU will use FXSAVE
```



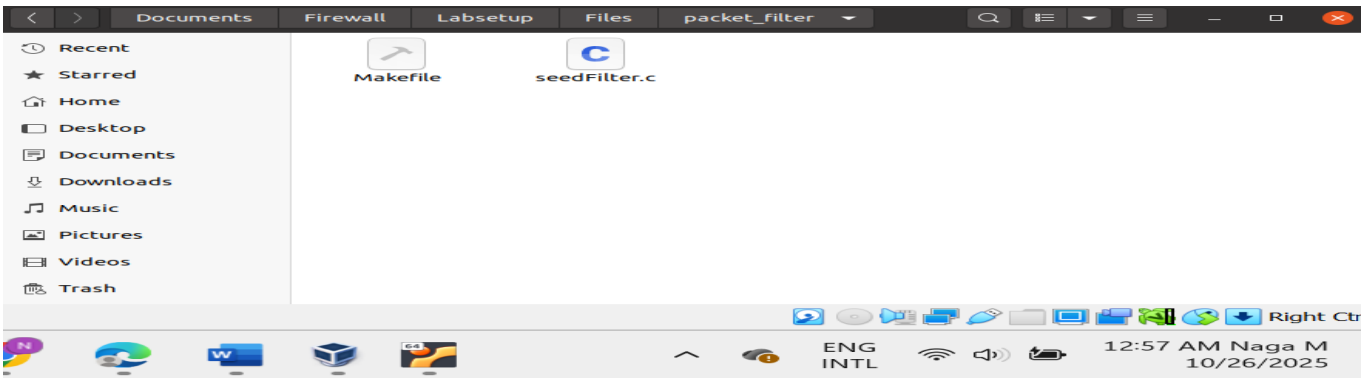

```
  382.351197] eth0: renamed from vethfc49c43
  382.370567] IPv6: ADDRCONF(NETDEV_CHANGE): veth02b8e72: link becomes ready
  382.370866] br-19c7345055d9: port 1(veth02b8e72) entered blocking state
  382.370869] br-19c7345055d9: port 1(veth02b8e72) entered forwarding state
  382.370976] IPv6: ADDRCONF(NETDEV_CHANGE): veth2fefa68: link becomes ready
  382.370997] br-4e96c1f6e002: port 4(veth2fefa68) entered blocking state
  382.370998] br-4e96c1f6e002: port 4(veth2fefa68) entered forwarding state
 2410.529702] hello: module verification failed: signature and/or required ke
issing - tainting kernel
 2410.539652] Hello World!
 2440.152524] Bye-bye World!.
10/26/25]seed@VM:~/.../kernel_module$ sudo dmesg
    0.000000] Linux version 5.4.0-54-generic (buildd@lcy01-amd64-024) (gcc ve
on 9.3.0 (Ubuntu 9.3.0-17ubuntu1~20.04)) #60-Ubuntu SMP Fri Nov 6 10:37:59 UT
020 (Ubuntu 5.4.0-54.60-generic 5.4.65)
    0.000000] Command line: BOOT_IMAGE=/boot/vmlinuz-5.4.0-54-generic root=UU
```



This successful completion shows we can compile, load, and unload kernel modules.

Task 1.B: Implement a Simple Firewall Using Net filter:

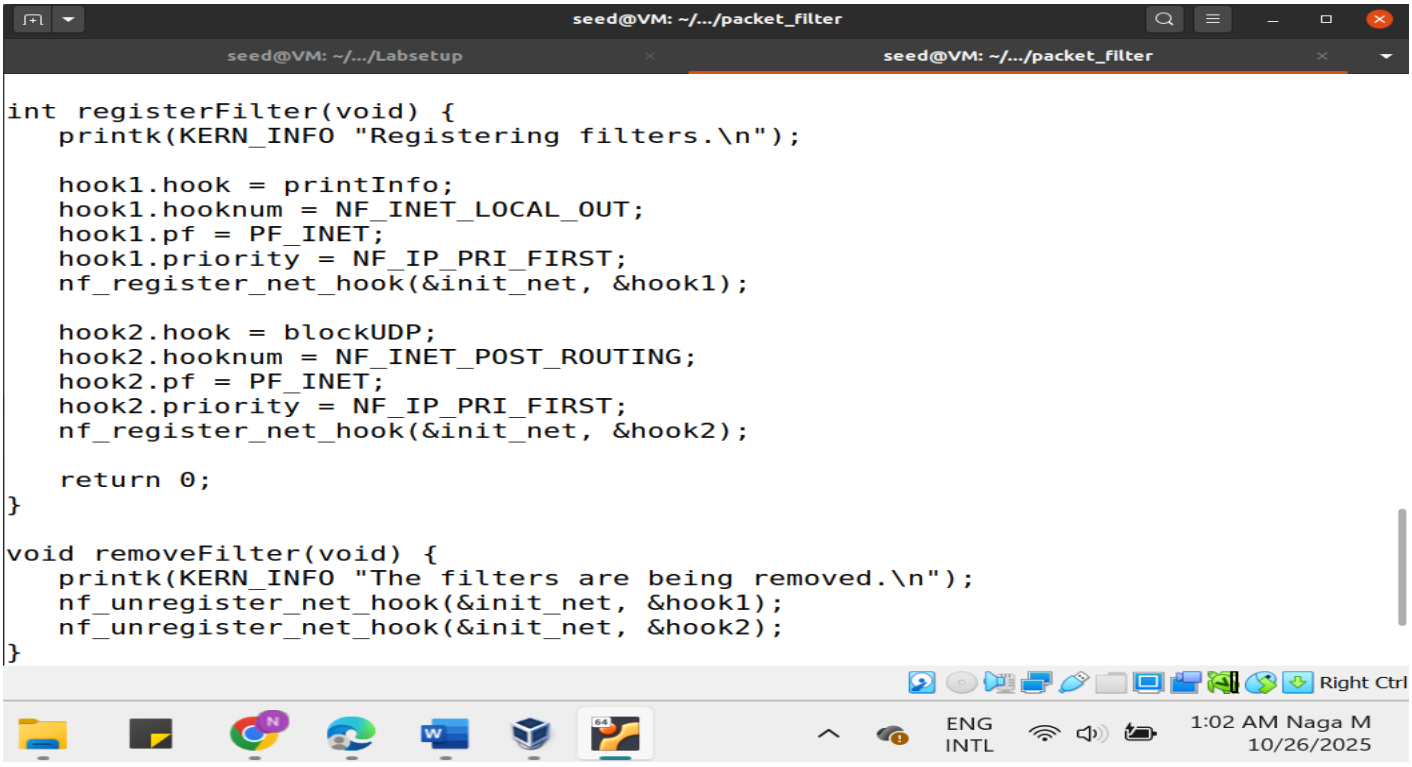
This task continues our main SEED VM (the host). This task is mainly to create a real firewall module that can block packets. For this task lets verify the complete sample code is called seedFilter.c, which is included in the lab setup files.



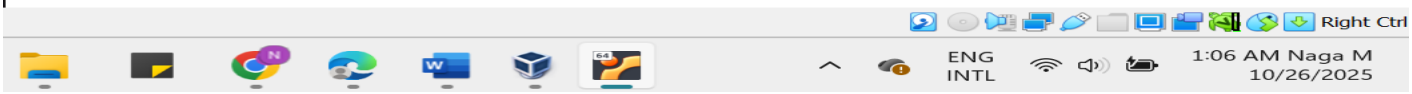
Task 1.B.(1):

: Block DNS: Block all DNS queries sent to Google's server (8.8.8.8).

For this task, lets compile the seedfilter.c file. Before that confirm that by using the command like **cat seedfilter.c**

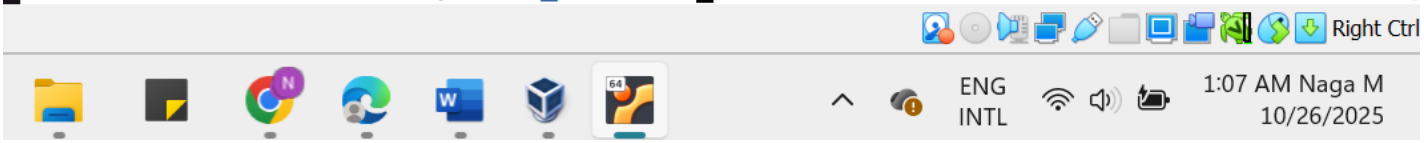


```
unsigned int blockUDP(void *priv, struct sk_buff *skb,
                      const struct nf_hook_state *state)
{
   struct iphdr *iph;
   struct udphdr *udph;

   u16  port   = 53;
   char ip[16] = "8.8.8.8";
   u32  ip_addr;

   if (!skb) return NF_ACCEPT;
```

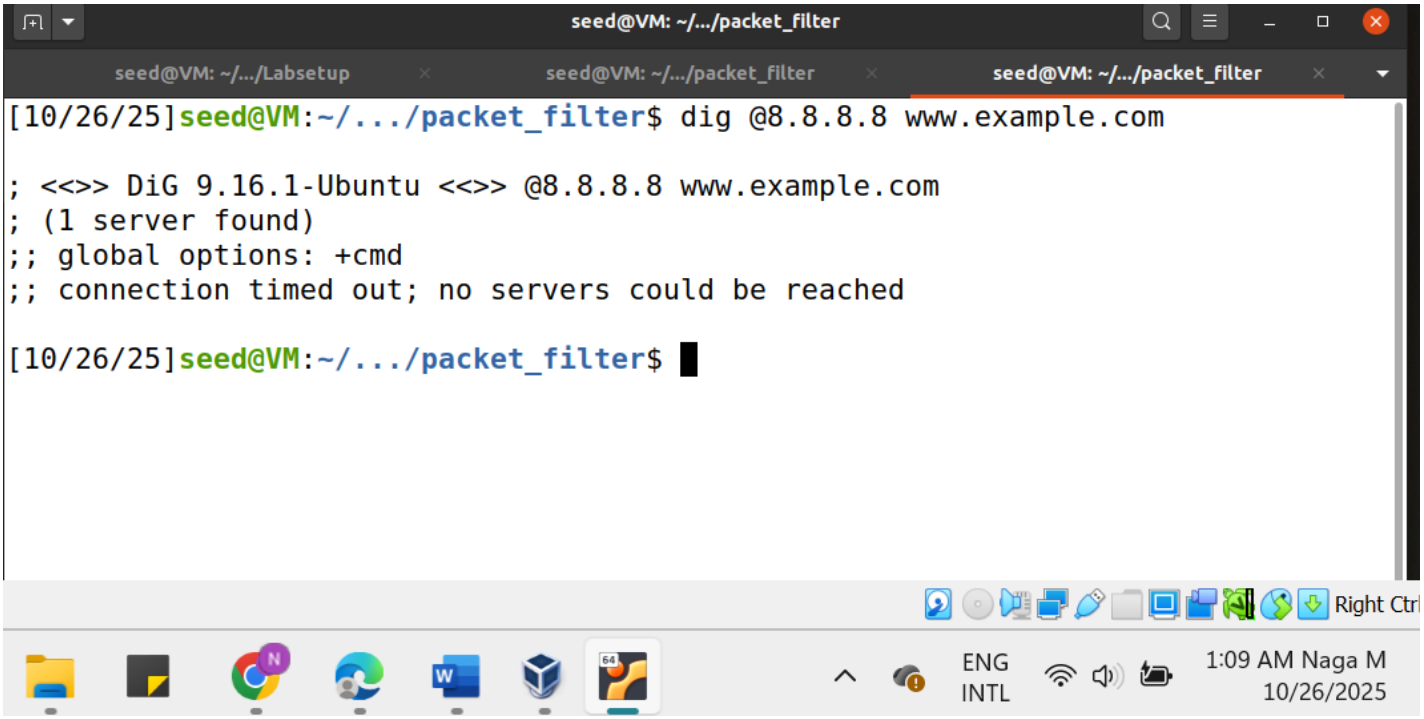


Lets run the all the commands,

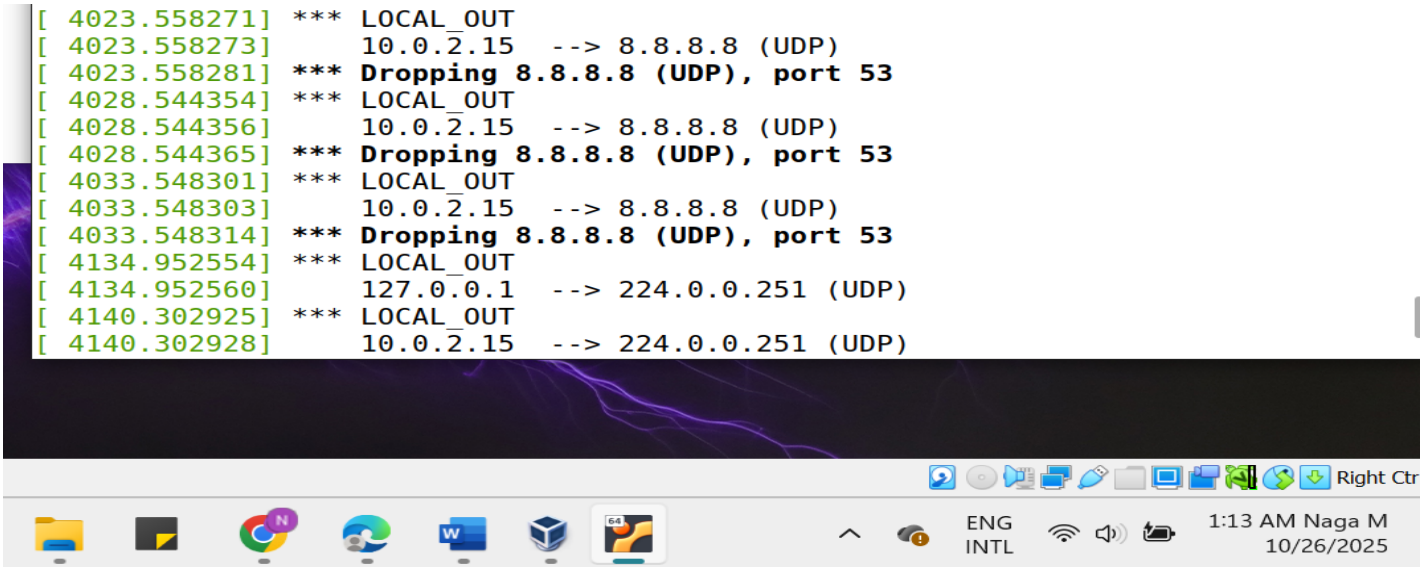
```
[10/26/25]seed@VM:~/.../packet_filter$ make
make -C /lib/modules/5.4.0-54-generic/build M=/home/seed/Documents/Firewall/Labs
etup/Files/packet_filter modules
make[1]: Entering directory '/usr/src/linux-headers-5.4.0-54-generic'
  CC [M]  /home/seed/Documents/Firewall/Labsetup/Files/packet_filter/seedFilter.
o
  Building modules, stage 2.
  MODPOST 1 modules
  CC [M]  /home/seed/Documents/Firewall/Labsetup/Files/packet_filter/seedFilter.
mod.o
  LD [M]  /home/seed/Documents/Firewall/Labsetup/Files/packet_filter/seedFilter.
ko
make[1]: Leaving directory '/usr/src/linux-headers-5.4.0-54-generic'
[10/26/25]seed@VM:~/.../packet_filter$ sudo insmod seedFilter.ko
[10/26/25]seed@VM:~/.../packet_filter$ lsmod | grep seedFilter
seedFilter             16384  0
[10/26/25]seed@VM:~/.../packet_filter$
```



Open the another terminal and type Open a second terminal on your SEED VM. Run the DNS query command. It will use 8.8.8.8 directly.



From the Output we observe that the command will hang for a long time and eventually fail with a "connection timed out" error because the query packet was dropped and no response was received. This means **the firewall is working.** By using the command dmesg in the terminal, we can confirm that Dropping 8.8.8.8



Unload the module before moving to the next sub-task. By using the command: sudo rmmod seedFilter

Task 1.B.(2):

: Experiment with Hooks

Lets Open seedFilter.c in a text editor, and add all the hooks. And compile, load the module.

```
[10/26/25]seed@VM:~/.../packet_filter$ nano seedFilter.c
[10/26/25]seed@VM:~/.../packet_filter$ make
make -C /lib/modules/5.4.0-54-generic/build M=/home/seed/Documents/Firewall/Labs
etup/Files/packet_filter modules
make[1]: Entering directory '/usr/src/linux-headers-5.4.0-54-generic'
  CC [M]  /home/seed/Documents/Firewall/Labsetup/Files/packet_filter/seedFilter.
o
  Building modules, stage 2.
  MODPOST 1 modules
  CC [M]  /home/seed/Documents/Firewall/Labsetup/Files/packet_filter/seedFilter.
mod.o
  LD [M]  /home/seed/Documents/Firewall/Labsetup/Files/packet_filter/seedFilter.
ko
make[1]: Leaving directory '/usr/src/linux-headers-5.4.0-54-generic'
[10/26/25]seed@VM:~/.../packet_filter$ ls -la seedFilter.ko
-rw-rw-r-- 1 seed seed 8392 Oct 26 02:40 seedFilter.ko
[10/26/25]seed@VM:~/.../packet_filter$ sudo insmod seedFilter.ko
[10/26/25]seed@VM:~/.../packet_filter$
```

Right Ctrl

ENG INTL 1:41 AM Naga M 10/26/2025

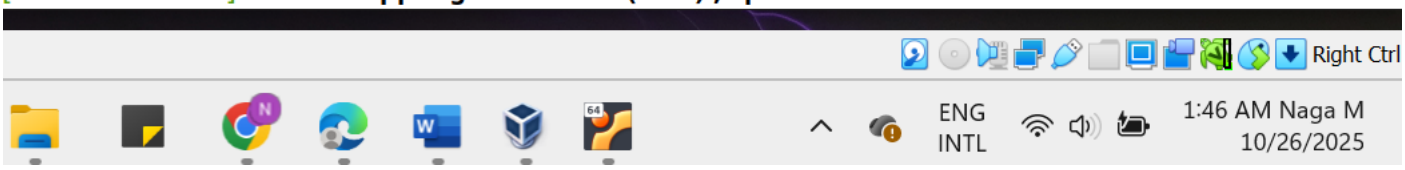
Generate different types of traffic and watch dmesg

```
[10/26/25]seed@VM:~/.../packet_filter$ dmesg
[    0.000000] Linux version 5.4.0-54-generic (buildd@lcy01-amd64-024) (gcc vers
ion 9.3.0 (Ubuntu 9.3.0-17ubuntu1~20.04)) #60-Ubuntu SMP Fri Nov 6 10:37:59 UTC
2020 (Ubuntu 5.4.0-54.60-generic 5.4.65)
[    0.000000] Command line: BOOT_IMAGE=/boot/vmlinuz-5.4.0-54-generic root=UUID
=a91f1a43-2770-4684-9fc3-b7abfd786c1d ro quiet splash
[    0.000000] KERNEL supported cpus:
[    0.000000]   Intel GenuineIntel
[    0.000000]   AMD AuthenticAMD
[    0.000000]   Hygon HygonGenuine
[    0.000000]   Centaur CentaurHauls
[    0.000000]   zhaoxin   Shanghai
[    0.000000] x86/fpu: x87 FPU will use FXSAVE
[    0.000000] BIOS-provided physical RAM map:
[    0.000000] BIOS-e820: [mem 0x0000000000000000-0x000000000009fbff] usable
[    0.000000] BIOS-e820: [mem 0x000000000009fc00-0x000000000009ffff] reserved
[    0.000000] BIOS-e820: [mem 0x00000000000f0000-0x00000000000fffff] reserved
[    0.000000] BIOS-e820: [mem 0x0000000000100000-0x000000007ffeffff] usable
[    0.000000] BIOS-e820: [mem 0x000000007fff0000-0x000000007fffffff] ACPI data
[    0.000000] BIOS-e820: [mem 0x00000000fec00000-0x00000000fec00fff] reserved
```

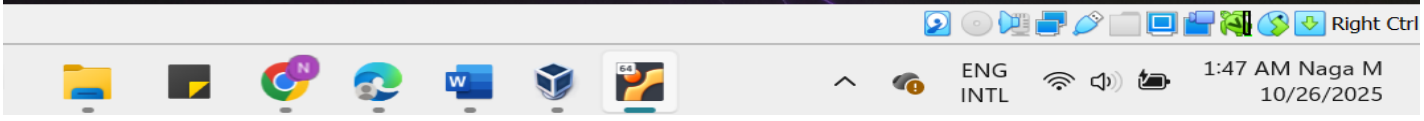
Right Ctrl

ENG INTL 1:44 AM Naga M 10/26/2025

```
[ 2410.539652] Hello World!
[ 2440.152524] Bye-bye World!.
[ 3904.283835] Registering filters.
[ 3947.086206] *** LOCAL_OUT
[ 3947.086212]     10.0.2.15   --> 192.168.1.1 (UDP)
[ 3947.120925] *** LOCAL_OUT
[ 3947.120931]     10.0.2.15   --> 91.189.91.48 (TCP)
[ 3947.171584] *** LOCAL_OUT
[ 3947.171758]     10.0.2.15   --> 91.189.91.48 (TCP)
[ 3947.172563] *** LOCAL_OUT
[ 3947.172565]     10.0.2.15   --> 91.189.91.48 (TCP)
[ 3947.224916] *** LOCAL_OUT
[ 3947.225802]     10.0.2.15   --> 91.189.91.48 (TCP)
[ 3947.227270] *** LOCAL_OUT
[ 3947.227274]     10.0.2.15   --> 91.189.91.48 (TCP)
[ 4023.556177] *** LOCAL_OUT
[ 4023.556181]     127.0.0.1   --> 127.0.0.1 (UDP)
[ 4023.558271] *** LOCAL_OUT
[ 4023.558273]     10.0.2.15   --> 8.8.8.8 (UDP)
[ 4023.558281] *** Dropping 8.8.8.8 (UDP), port 53
[ 4028.544354] *** LOCAL_OUT
[ 4028.544356]     10.0.2.15   --> 8.8.8.8 (UDP)
[ 4028.544365] *** Dropping 8.8.8.8 (UDP), port 53
```




```
[ 4398.574783] *** LOCAL_OUT
[ 4398.574786]     10.0.2.15   --> 199.232.70.49 (TCP)
[ 4398.574857] *** LOCAL_OUT
[ 4398.574858]     10.0.2.15   --> 199.232.70.49 (TCP)
[ 4431.036163] The filters are being removed.
[ 5952.514464] Registering filters to all five Netfilter hooks.
[ 5952.514473] All five Netfilter hooks registered successfully.
[ 5957.197407] *** [LOCAL_OUT] 10.0.2.15:UDP --> 192.168.1.1:UDP
[ 5957.197417] *** [POST_ROUTING] 10.0.2.15:UDP --> 192.168.1.1:UDP
[ 5957.229231] *** [PRE_ROUTING] 192.168.1.1:UDP --> 10.0.2.15:UDP
[ 5957.229663] *** [LOCAL_IN] 192.168.1.1:UDP --> 10.0.2.15:UDP
[ 5981.632127] *** [LOCAL_OUT] 127.0.0.1:UDP --> 127.0.0.53:UDP
[ 5981.632146] *** [POST_ROUTING] 127.0.0.1:UDP --> 127.0.0.53:UDP
[ 5981.632292] *** [PRE_ROUTING] 127.0.0.1:UDP --> 127.0.0.53:UDP
```

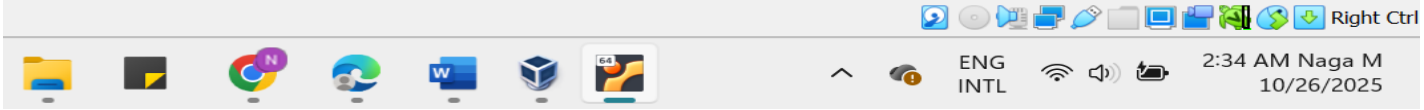


From this results we can see that the packet flow: LOCAL_OUT → POST_ROUTING → PRE_ROUTING → LOCAL_IN.

Local_OUT: when machine create packets, POST_ROUTING: Right BEFORE packets leave our machine, PRE_ROUTING: Right AFTER packets arrive at our machine, LOCAL_IN : When packets are destined FOR OUR machine. In this Forward was not triggered because here Packets were not passing through your machine to other destinations.

Task 1.B.(3):

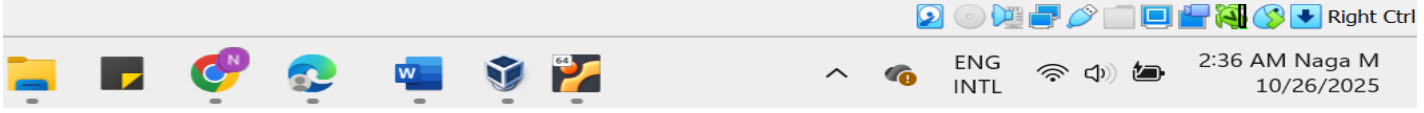
```
[10/26/25]seed@VM:~/.../packet_filter$ dockps
0af169a8c0d8  hostA-10.9.0.5
113e6905030e  host1-192.168.60.5
232e35842dce  seed-router
4429371358b1  host2-192.168.60.6
566c77e9ddfe  host3-192.168.60.7
[10/26/25]seed@VM:~/.../packet_filter$ docksh 0af
root@0af169a8c0d8:/# ping 10.9.0.1
PING 10.9.0.1 (10.9.0.1) 56(84) bytes of data.
64 bytes from 10.9.0.1: icmp_seq=1 ttl=64 time=0.152 ms
64 bytes from 10.9.0.1: icmp_seq=2 ttl=64 time=0.062 ms
64 bytes from 10.9.0.1: icmp_seq=3 ttl=64 time=0.103 ms
64 bytes from 10.9.0.1: icmp_seq=4 ttl=64 time=0.126 ms
64 bytes from 10.9.0.1: icmp_seq=5 ttl=64 time=0.104 ms
64 bytes from 10.9.0.1: icmp_seq=6 ttl=64 time=0.237 ms
^Z
[1]+  Stopped                 ping 10.9.0.1
root@0af169a8c0d8:/# ping 10.9.0.1
PING 10.9.0.1 (10.9.0.1) 56(84) bytes of data.
```



seed@VM: ~/.../packet_filter

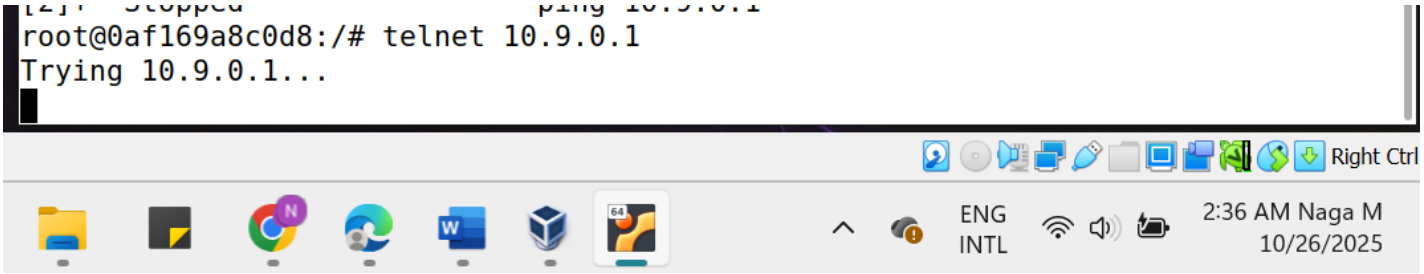
seed@VM: ~/.../Labsetup | seed@VM: ~/.../packet_fi... | seed@VM: ~/.../packet_fi... | seed@VM: ~/.../packet_fi...

```
[ 9238.252304] *** PRE_ROUTING: 10.9.0.5 --> 10.9.0.1 (ICMP)
[ 9238.252308] *** LOCAL_IN: 10.9.0.5 --> 10.9.0.1 (ICMP)
[ 9238.252311] *** BLOCKING PING to VM: 10.9.0.5 --> 10.9.0.1 (ICMP Echo Request
)
[ 9239.276258] *** PRE_ROUTING: 10.9.0.5 --> 10.9.0.1 (ICMP)
[ 9239.276273] *** PRE_ROUTING: 10.9.0.5 --> 10.9.0.1 (ICMP)
[ 9239.276277] *** LOCAL_IN: 10.9.0.5 --> 10.9.0.1 (ICMP)
[ 9239.276280] *** BLOCKING PING to VM: 10.9.0.5 --> 10.9.0.1 (ICMP Echo Request
)
[ 9240.299917] *** PRE_ROUTING: 10.9.0.5 --> 10.9.0.1 (ICMP)
[ 9240.299933] *** PRE_ROUTING: 10.9.0.5 --> 10.9.0.1 (ICMP)
[ 9240.299936] *** LOCAL_IN: 10.9.0.5 --> 10.9.0.1 (ICMP)
[ 9240.299938] *** BLOCKING PING to VM: 10.9.0.5 --> 10.9.0.1 (ICMP Echo Request
)
[ 9241.323993] *** PRE_ROUTING: 10.9.0.5 --> 10.9.0.1 (ICMP)
[ 9241.324004] *** PRE_ROUTING: 10.9.0.5 --> 10.9.0.1 (ICMP)
[ 9241.324008] *** LOCAL_IN: 10.9.0.5 --> 10.9.0.1 (ICMP)
[ 9241.324009] *** BLOCKING PING to VM: 10.9.0.5 --> 10.9.0.1 (ICMP Echo Request
)
[ 9257.088392] *** LOCAL_OUT: 10.0.2.15 --> 192.168.1.1 (UDP)
[ 9257.088415] *** POST_ROUTING: 10.0.2.15 --> 192.168.1.1 (UDP)
[ 9257.117305] *** PRE_ROUTING: 192.168.1.1 --> 10.0.2.15 (UDP)
[ 9257.117805] *** LOCAL_IN: 192.168.1.1 --> 10.0.2.15 (UDP)
[10/26/25]seed@VM:~/.../packet_filter$
```

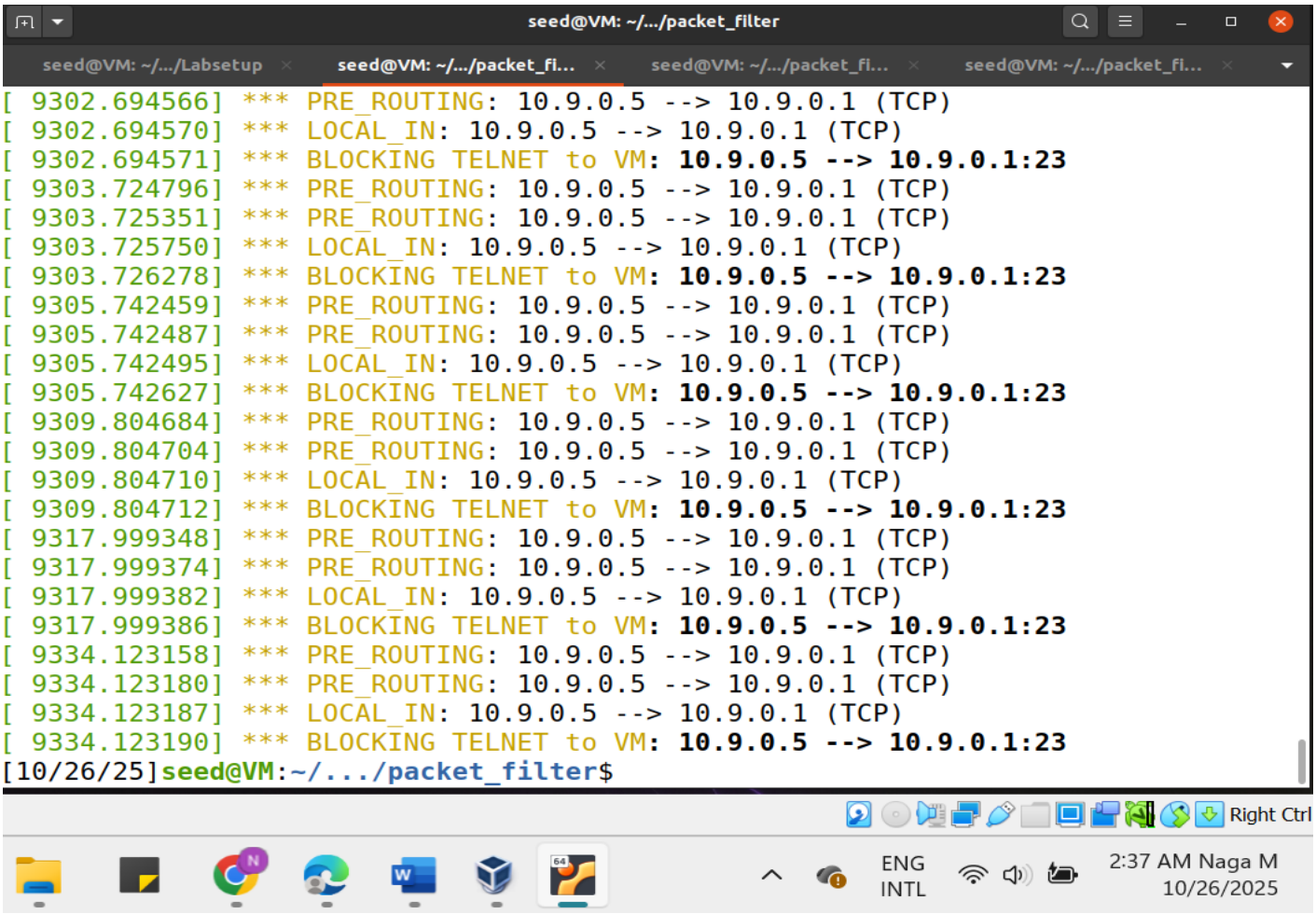


Our task was successful. From this output we can observe that Ping request was blocked. ICMP packets from 10.9.0.5 → 10.9.0.1 are caught at LOCAL_IN hook and then our block PING function successfully identifies and drops ICMP Echo Requests.

For Telnet, From the host-A container we can try to connect the telnet 10.9.0.1, the connection was not established.



We can see the result as below that the telnet connection was blocked. That means the attack was successful.



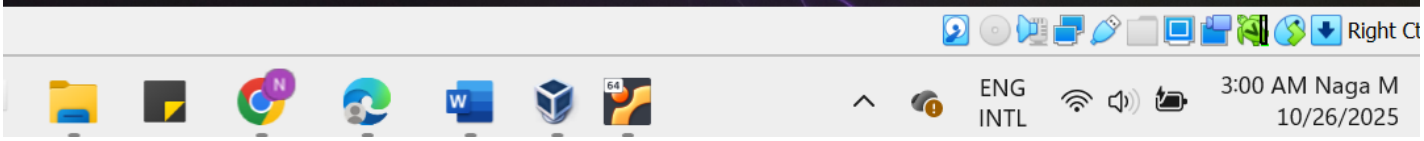
TELNET BLOCKING was also WORKING. TCP port 23 packets from 10.9.0.5 → 10.9.0.1 are caught at LOCAL_IN hook and then our blockTELNET function successfully blocks telnet connection.

Task 2: Experimenting with Stateless Firewall Rules

In this Task, we Learn how to configure a firewall using iptables commands to create stateless rules how to construct rules to control packet flow based on source/destination IPs, ports, protocols, and network interfaces..

Task 2.A: Protecting the Router

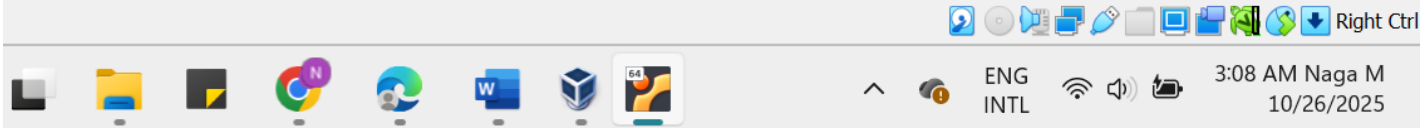
```
root@232e35842dce:/# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group defaul
t qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
16: eth0@if17: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP
 group default
    link/ether 02:42:0a:09:00:0b brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.9.0.11/24 brd 10.9.0.255 scope global eth0
       valid_lft forever preferred_lft forever
18: eth1@if19: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP
 group default
    link/ether 02:42:c0:a8:3c:0b brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 192.168.60.11/24 brd 192.168.60.255 scope global eth1
       valid_lft forever preferred_lft forever
root@232e35842dce:/#
```



From this we can see that the router ip address is : 10.9.0.11

Open the seed-router container and run the iptables commands

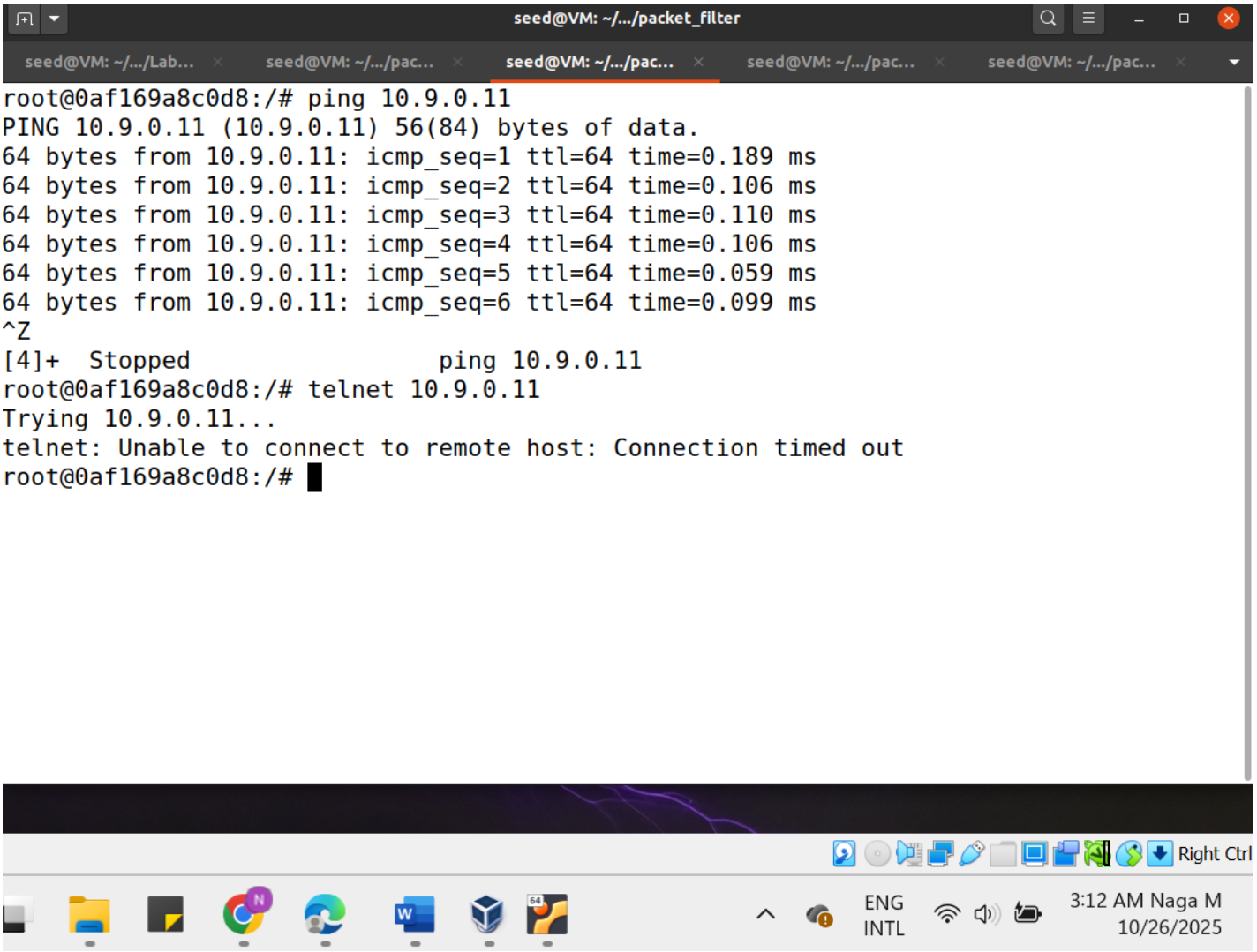
```
root@232e35842dce:/# iptables -A INPUT -p icmp --icmp-type echo-request -j ACCEPT
root@232e35842dce:/# iptables -A OUTPUT -p icmp --icmp-type echo-reply -j ACCEPT
root@232e35842dce:/# iptables -P OUTPUT DROP
root@232e35842dce:/# iptables -P INPUT DROP
root@232e35842dce:/#
```



From the host-A container try to ping 10.9.0.11, we will see that will work whereas the telnet 10.9.0.11 will not. The Lab result is shown in below.

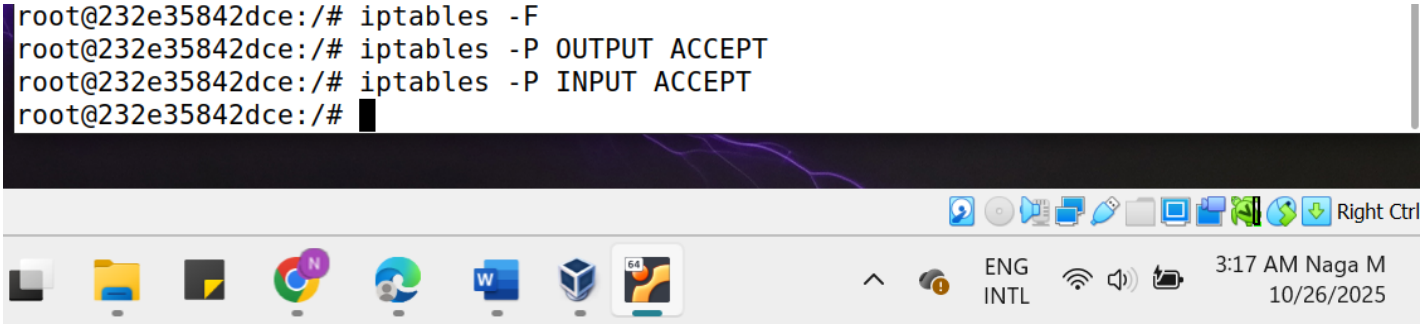
Why Ping works: The first rule allows ping requests INTO the router and, the second rule allows ping responses OUT of the router. We should see successful ping responses and then packet transfer.

Why Telnet was not work: There are no rules allowing TCP traffic on port 23 (telnet). The default INPUT DROP policy blocks all incoming telnet connections. So, the connection was timeout.

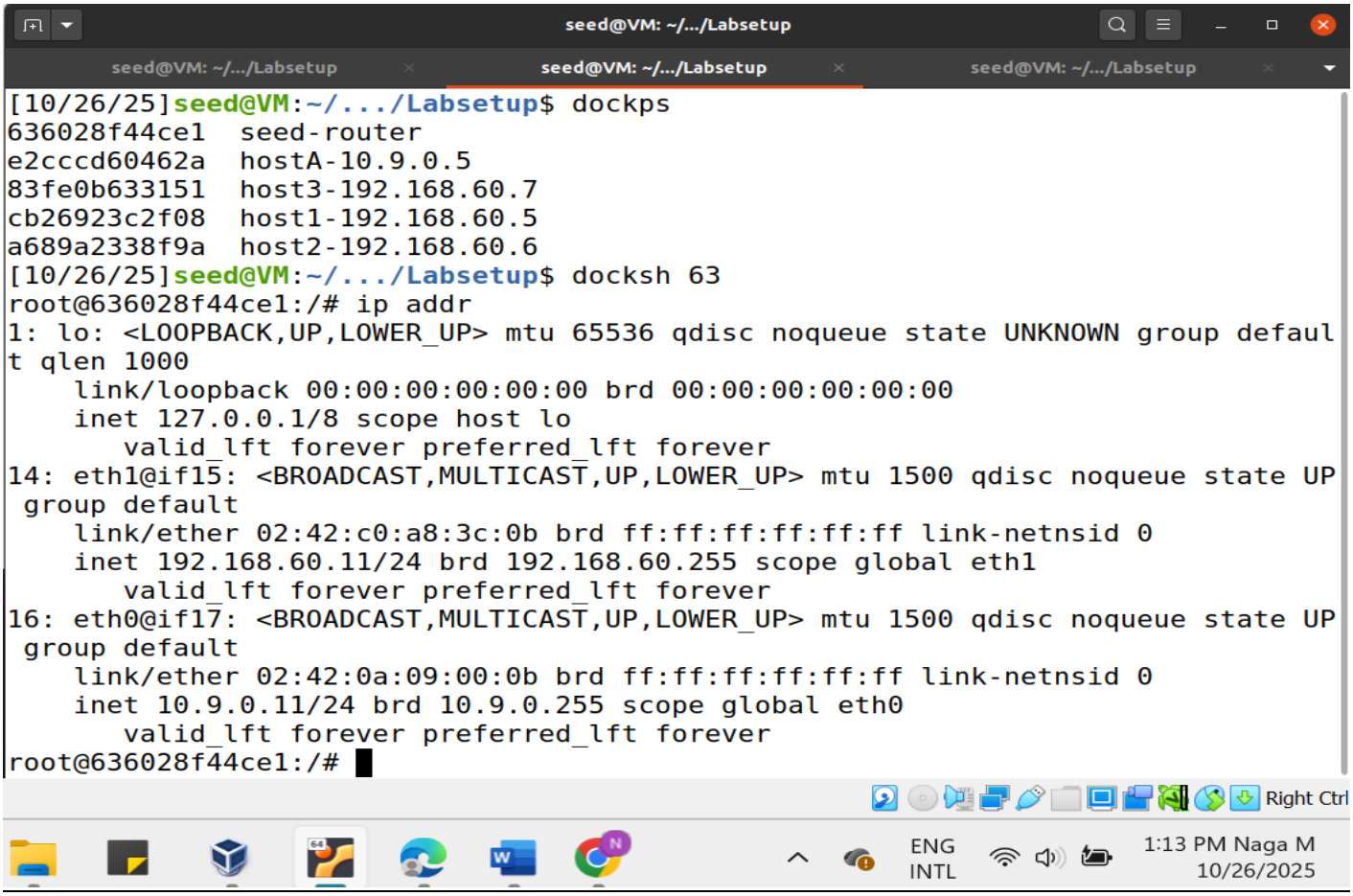


As per my observation, The firewall rules successfully Allowed ping traffic (ICMP echo request/reply). Blocked all other incoming traffic including telnet. That means it implemented a secure default-deny policy.

Before moving to the next Task Cleanup is very important.



Task 2.B: Protecting the Internal Network: we'll need to first identify the interfaces on the router



Here, we can see that

- eth0: 10.9.0.11/24 (External network interface)
- eth1: 192.168.60.11/24 (Internal network interface)

In this task, we want to implement a firewall to protect the internal network. More specifically, we need to enforce the following RULE restrictions on the ICMP traffic:

1. Allow internal hosts to ping outside (echo-request going out)

iptables -A FORWARD -i eth1 -o eth0 -p icmp --icmp-type echo-request -j ACCEPT

2. Allow ping responses back to internal hosts (echo-reply coming in)

iptables -A FORWARD -i eth0 -o eth1 -p icmp --icmp-type echo-reply -j ACCEPT

3. Block outside hosts from pinging internal hosts

iptables -A FORWARD -i eth0 -o eth1 -p icmp --icmp-type echo-request -j DROP

4. Set default FORWARD policy to DROP (blocks all other traffic between networks)

iptables -P FORWARD DROP

Lets execute the commands in this order only…

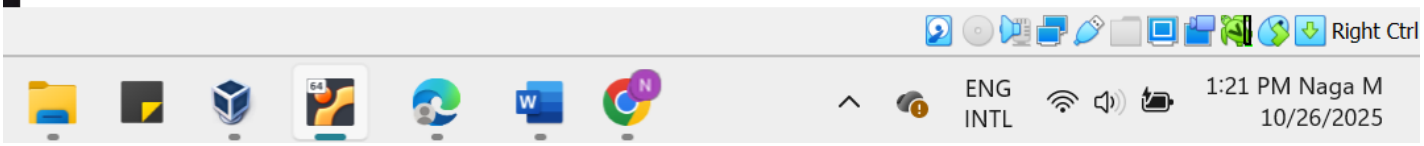
```
root@636028f44ce1:/# iptables -A FORWARD -i eth1 -o eth0 -p icmp --icmp-type echo-request -j ACCEPT
root@636028f44ce1:/# iptables -A FORWARD -i eth0 -o eth1 -p icmp --icmp-type echo-reply -j ACCEPT
root@636028f44ce1:/# iptables -A FORWARD -i eth0 -o eth1 -p icmp --icmp-type echo-request -j DROP
root@636028f44ce1:/# iptables -P FORWARD DROP
root@636028f44ce1:/#
```

And then lets try the conditions:

1. Internal host pings outside host (Should WORK)

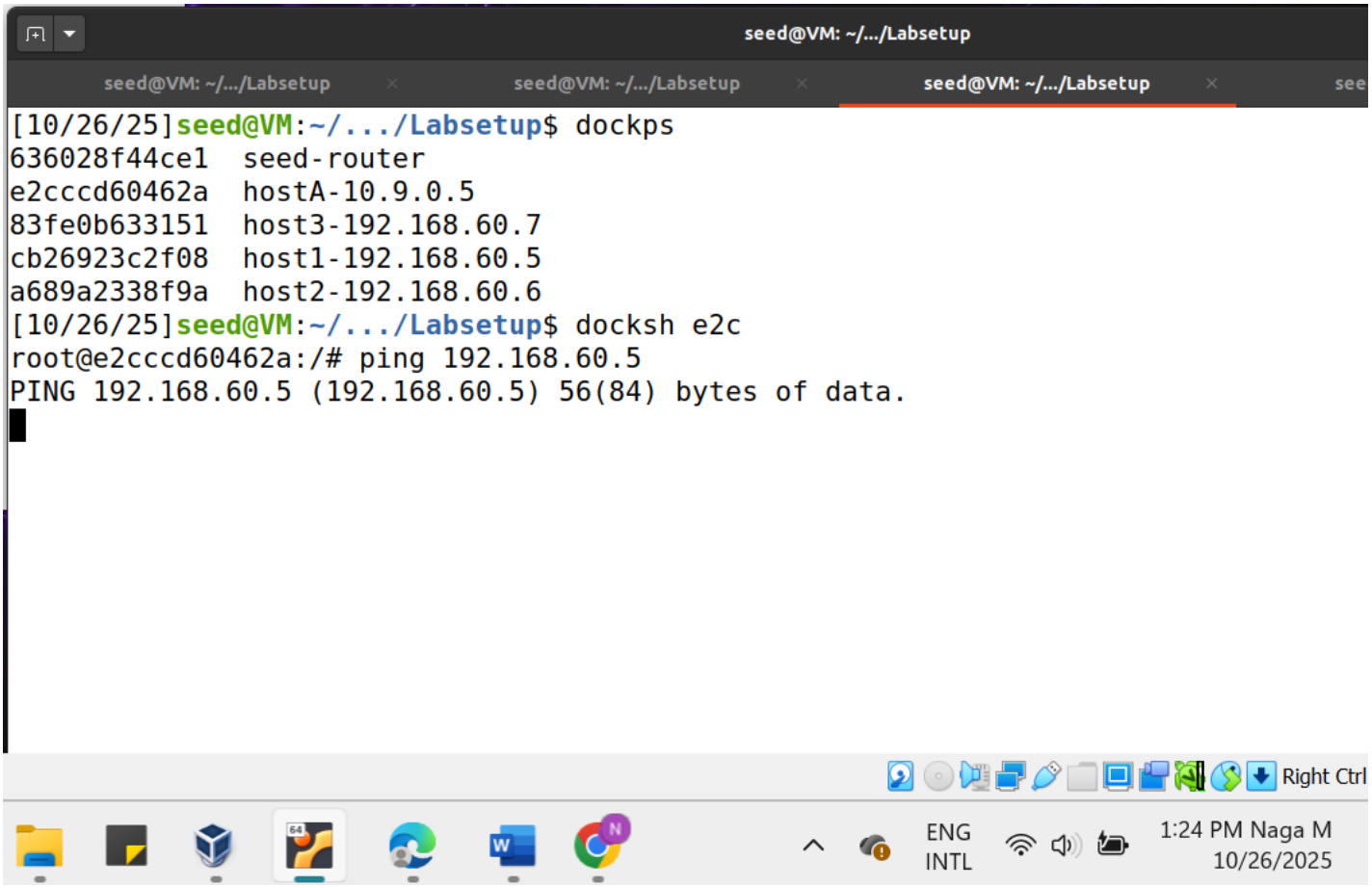
Lets ping 10.9.0.5 from any one of the internal host lets take 192.168.60.5 , Here is the result… it was worked.

```
[10/26/25]seed@VM:~/.../Labsetup$ dockps
636028f44ce1  seed-router
e2cccd60462a  hostA-10.9.0.5
83fe0b633151  host3-192.168.60.7
cb26923c2f08  host1-192.168.60.5
a689a2338f9a  host2-192.168.60.6
[10/26/25]seed@VM:~/.../Labsetup$ docksh cb2
root@cb26923c2f08:/# ping 10.9.0.5
PING 10.9.0.5 (10.9.0.5) 56(84) bytes of data.
64 bytes from 10.9.0.5: icmp_seq=1 ttl=63 time=0.121 ms
64 bytes from 10.9.0.5: icmp_seq=2 ttl=63 time=0.127 ms
64 bytes from 10.9.0.5: icmp_seq=3 ttl=63 time=0.126 ms
64 bytes from 10.9.0.5: icmp_seq=4 ttl=63 time=0.133 ms
64 bytes from 10.9.0.5: icmp_seq=5 ttl=63 time=0.129 ms
64 bytes from 10.9.0.5: icmp_seq=6 ttl=63 time=0.125 ms
^Z
[1]+  Stopped                 ping 10.9.0.5
root@cb26923c2f08:/#
```



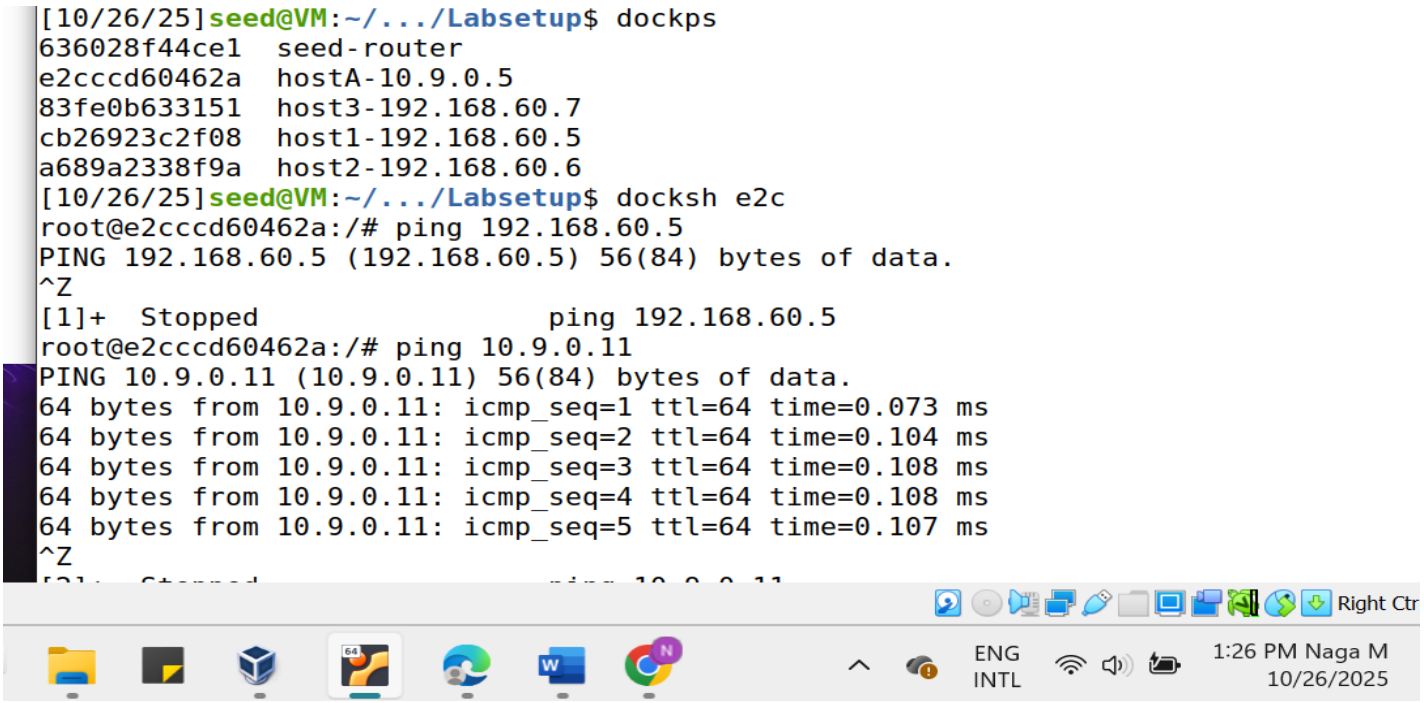
2. Outside hosts cannot ping internal hosts

Lets try to ping the 192.168.60.5 from the 10.9.0.5… it was failed that means our firewall setting was worked successfully.



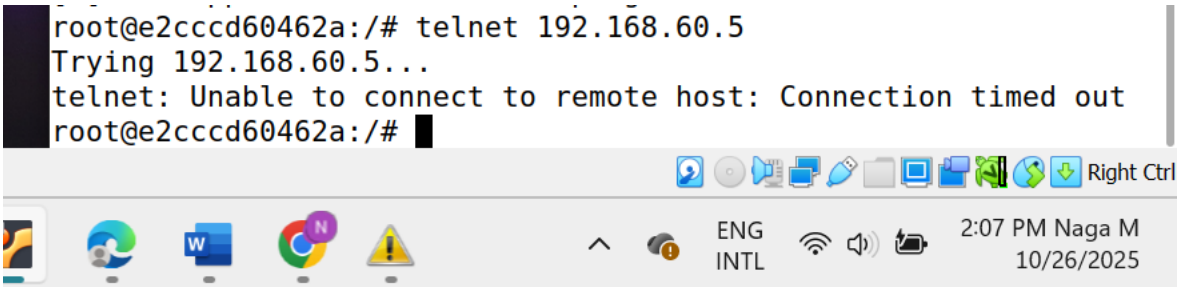
3. Outside hosts can ping the router.

Lets try to ping the 190.90.11 from the hst-A(10.9.0.5).. the ping command was worked that mean out rule for firewall was successful.

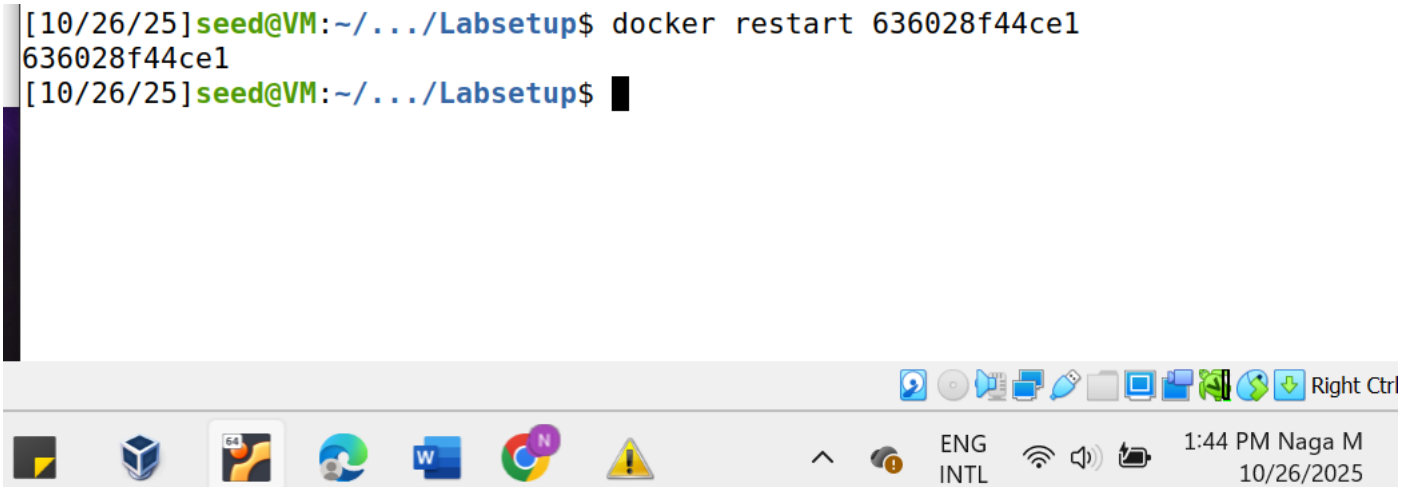


4. All other packets between the internal and external networks should be blocked.

Lets try telnet from host-A(10.9.0.5)…. It was failed means our firewall setting was successful.

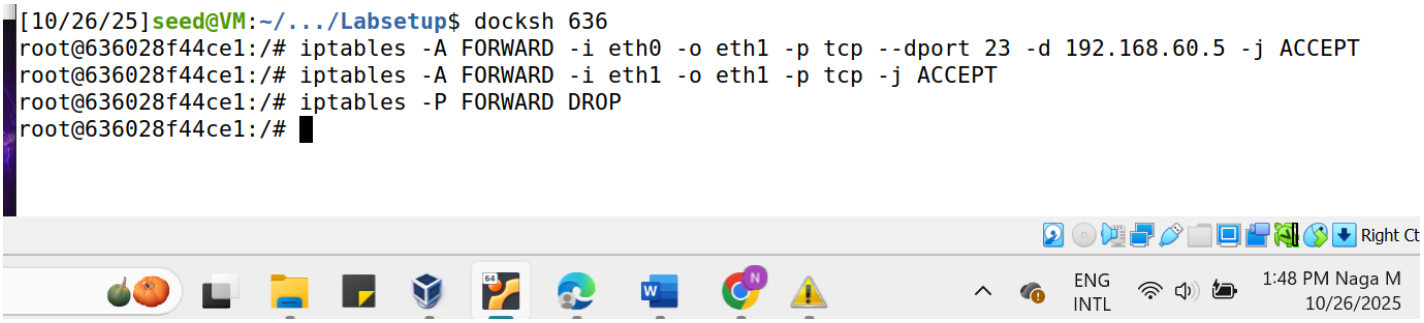


Cleanup: clean the table or restart the container before moving on to the next task



Task 2.C: Protecting Internal Servers:

Execute these commands on the router container:



All the internal hosts run a telnet server (listening to port 23). Outside hosts can only access the telnet server on 192.168.60.5, not the other internal hosts.

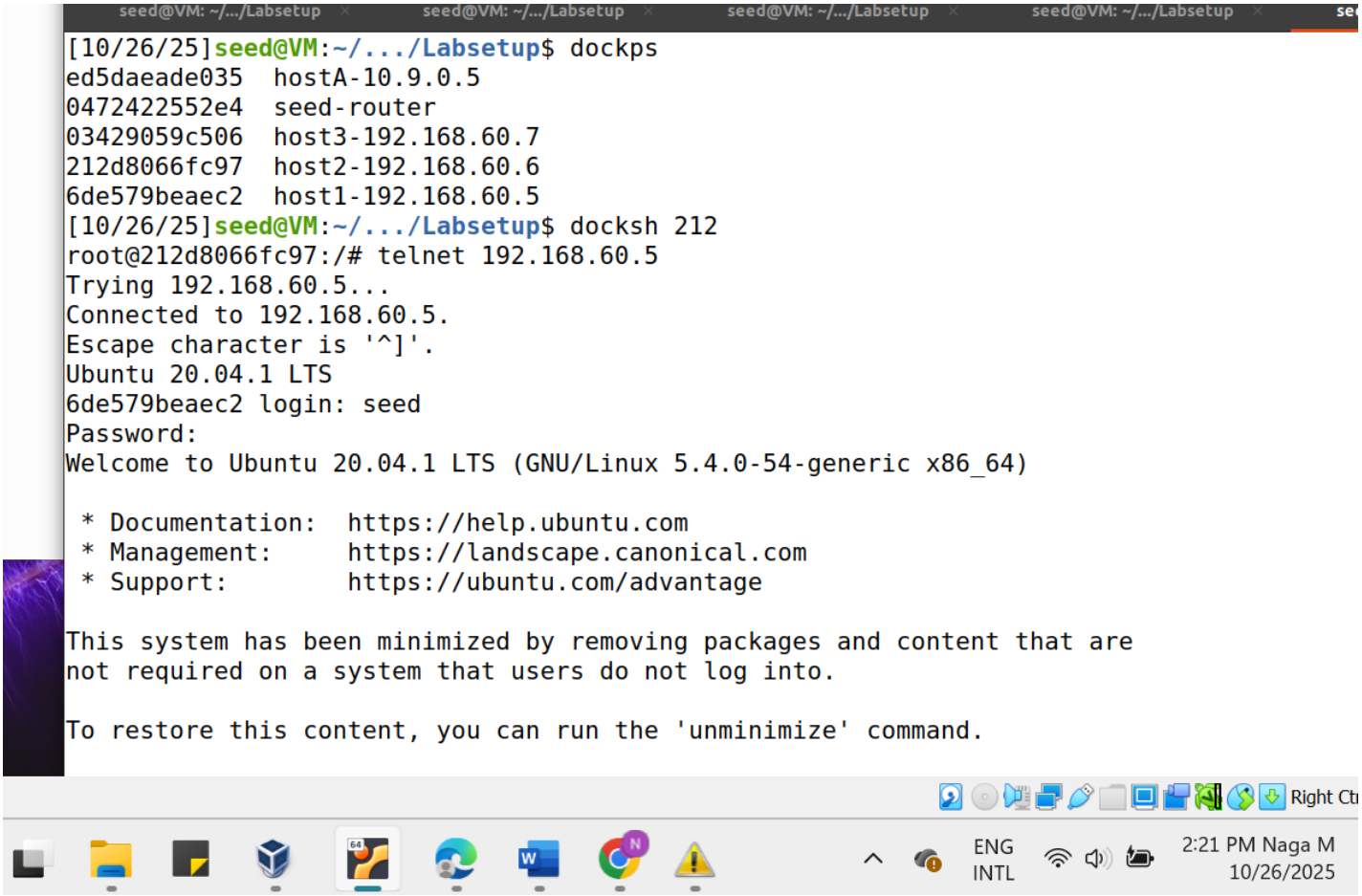
```
root@51da5bddeb45:/# telnet 192.168.60.5
Trying 192.168.60.5...
Connected to 192.168.60.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
011d53efb0d7 login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
```

Note: My system got crashed so then I rebuild the container..

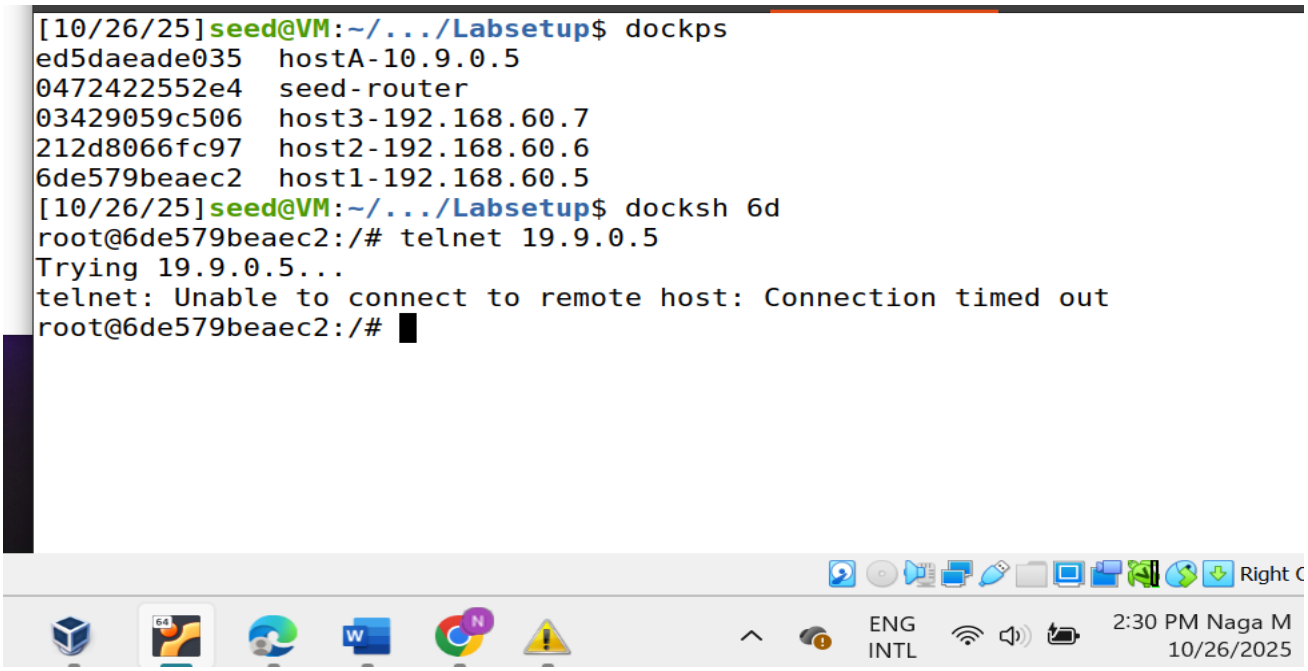
2. Outside hosts cannot access other internal server: From the outside host(10.9.0.5) try to telnet the 192.168.60.6., 192.168.60.7…the connection was not established mean the firewall setting was successful.

```
root@ed5daeade035:/# telnet 192.168.60.6
Trying 192.168.60.6...
telnet: Unable to connect to remote host: Connection timed out
root@ed5daeade035:/# telnet 192.168.60.7
Trying 192.168.60.7...
telnet: Unable to connect to remote host: Connection timed out
root@ed5daeade035:/#
```

3. Internal hosts can access all the internal servers. (telnet 192.168.60.5 from the Host-2).. connection was established means firewall setting was successful.



4. Internal hosts cannot access external servers. From the host-1(192.168.60.5) try to telnet 10.9.0.5.. the connection was failed that means our firewall setting was successful.

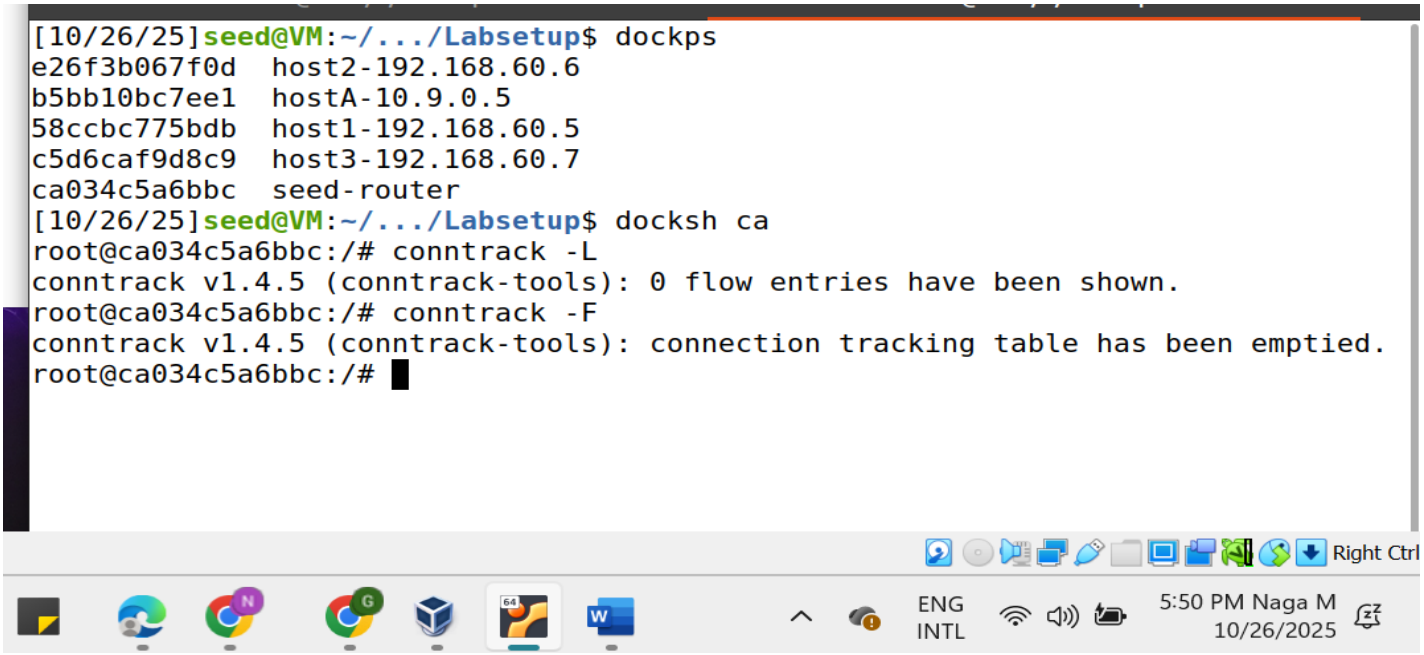


Task3: Connection Tracking and Stateful Firewall

This task moves from stateless packet filtering (inspecting each packet in isolation) to stateful filtering (understanding packets in the context of a connection).

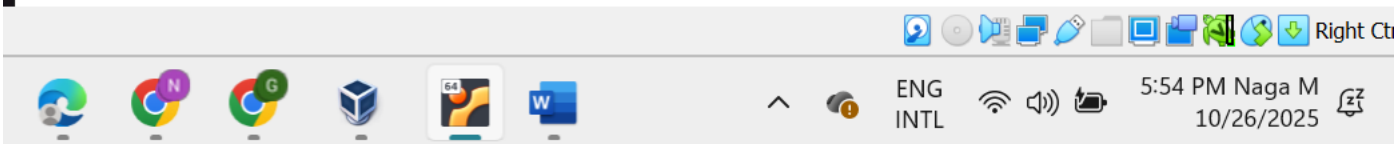
Task 3.A: Experiment with the Connection Tracking

On the Router container, start monitoring the connection track table. First, clear any old entries to have a clean slate.



ICMP experiment: : From Host A (10.9.0.5), ping an **internal host (192.168.60.5)**. Let it run for a few packets. Ans then move on the router container and lets run the command conntrack -L

```
[10/26/25]seed@VM:~/.../Labsetup$ docksh b5
root@b5bb10bc7ee1:/#  ping 192.168.60.5
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data.
64 bytes from 192.168.60.5: icmp_seq=1 ttl=63 time=0.417 ms
64 bytes from 192.168.60.5: icmp_seq=2 ttl=63 time=0.116 ms
64 bytes from 192.168.60.5: icmp_seq=3 ttl=63 time=0.383 ms
64 bytes from 192.168.60.5: icmp_seq=4 ttl=63 time=0.485 ms
64 bytes from 192.168.60.5: icmp_seq=5 ttl=63 time=0.071 ms
64 bytes from 192.168.60.5: icmp_seq=6 ttl=63 time=0.078 ms
64 bytes from 192.168.60.5: icmp_seq=7 ttl=63 time=0.106 ms
64 bytes from 192.168.60.5: icmp_seq=8 ttl=63 time=0.120 ms
[10/26/25]seed@VM:~/.../Labsetup$ dockps
e26f3b067f0d  host2-192.168.60.6
b5bb10bc7ee1  hostA-10.9.0.5
58ccbc775bdb  host1-192.168.60.5
c5d6caf9d8c9  host3-192.168.60.7
ca034c5a6bbc  seed-router
[10/26/25]seed@VM:~/.../Labsetup$ docksh ca
root@ca034c5a6bbc:/# conntrack -L
conntrack v1.4.5 (conntrack-tools): 0 flow entries have been shown.
root@ca034c5a6bbc:/# conntrack -F
conntrack v1.4.5 (conntrack-tools): connection tracking table has been emptied.
root@ca034c5a6bbc:/# conntrack -L
icmp     1 29 src=10.9.0.5 dst=192.168.60.5 type=8 code=0 id=28 src=192.168.60.5
 dst=10.9.0.5 type=0 code=0 id=28 mark=0 use=1
conntrack v1.4.5 (conntrack-tools): 1 flow entries have been shown.
root@ca034c5a6bbc:/#
```



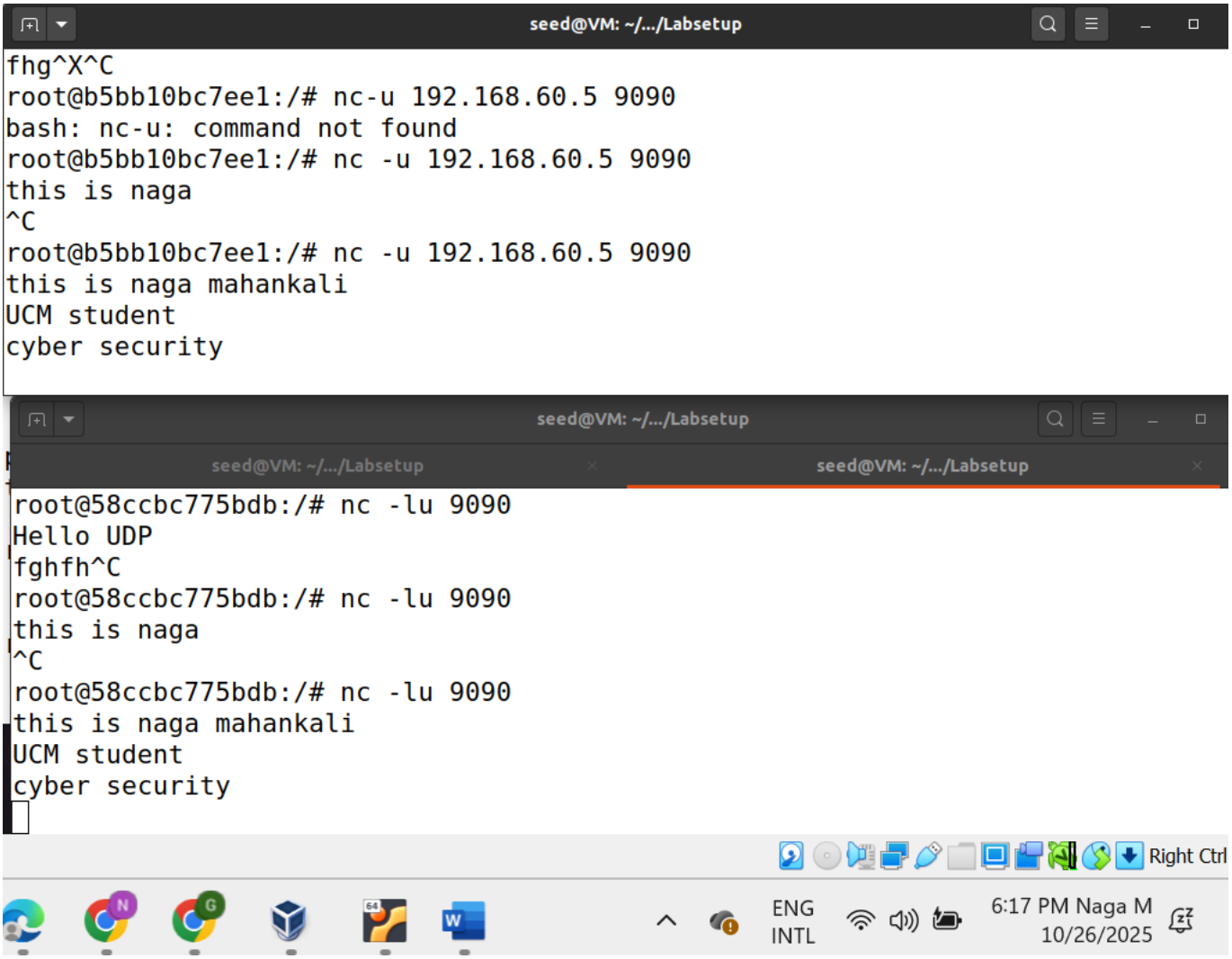
- How long is the ICMP connection state be kept: Based on the conntrack -L output, the ICMP connection state will be kept for **29 seconds**. 1 = Seconds remaining until expiration (constantly refreshes with activity). 29 = Total timeout value for ICMP connections

My observations: The 29-second timer starts when the ICMP conversation begins. Each new ICMP packet (ping request/reply) resets the timer back to 29 seconds .If no ICMP packets are exchanged for 29 seconds, the connection state entry is automatically removed from the tracking table

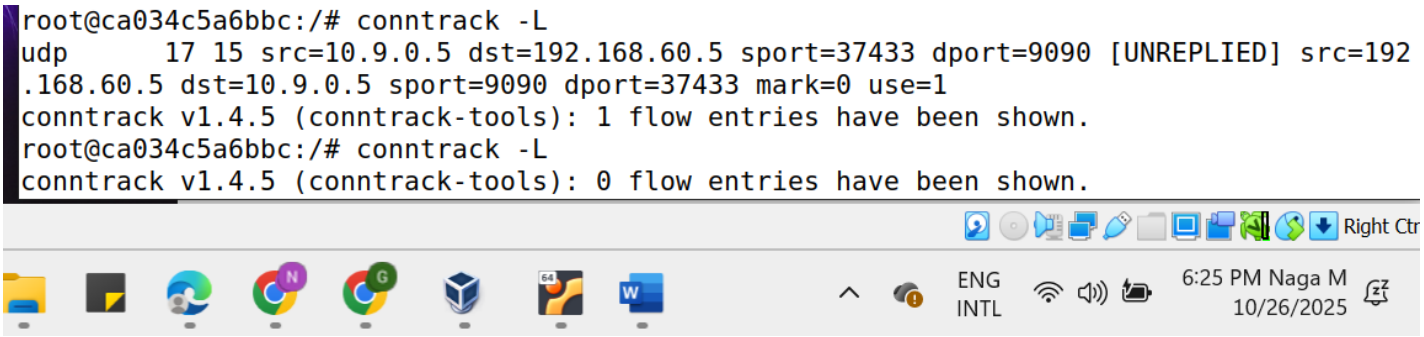
UDPexperiment:

On the Router container, flush the connection table again. And then **On an internal host (e.g., 192.168.60.5)**, start a UDP server. Get a shell on it from a new terminal.

On 192.168.60.5, start a netcat UDP server # nc-lu 9090 // On 10.9.0.5, send out UDP packets # nc-u 192.168.60.5 9090



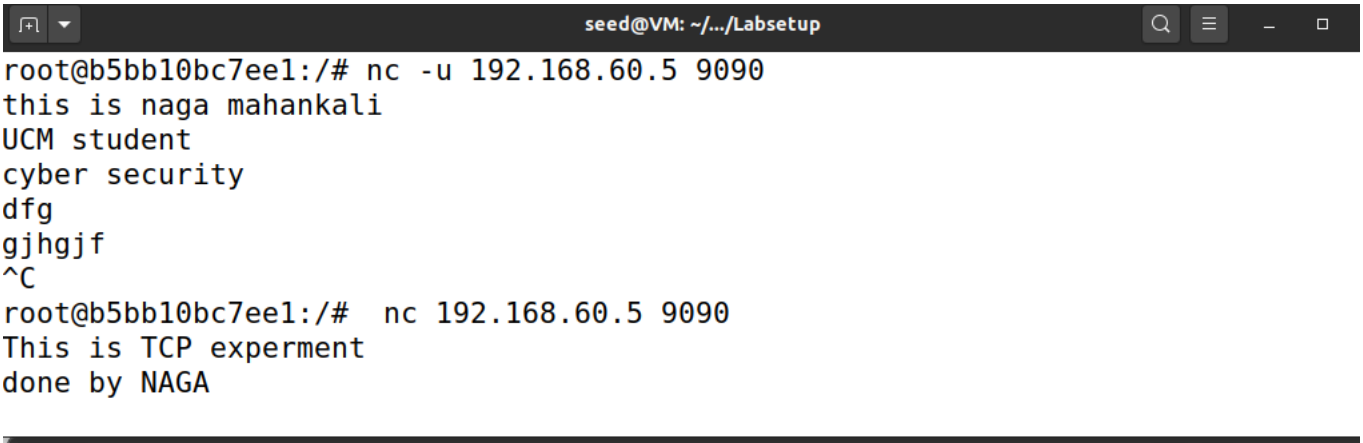
Check the connection table on the **Router container**.

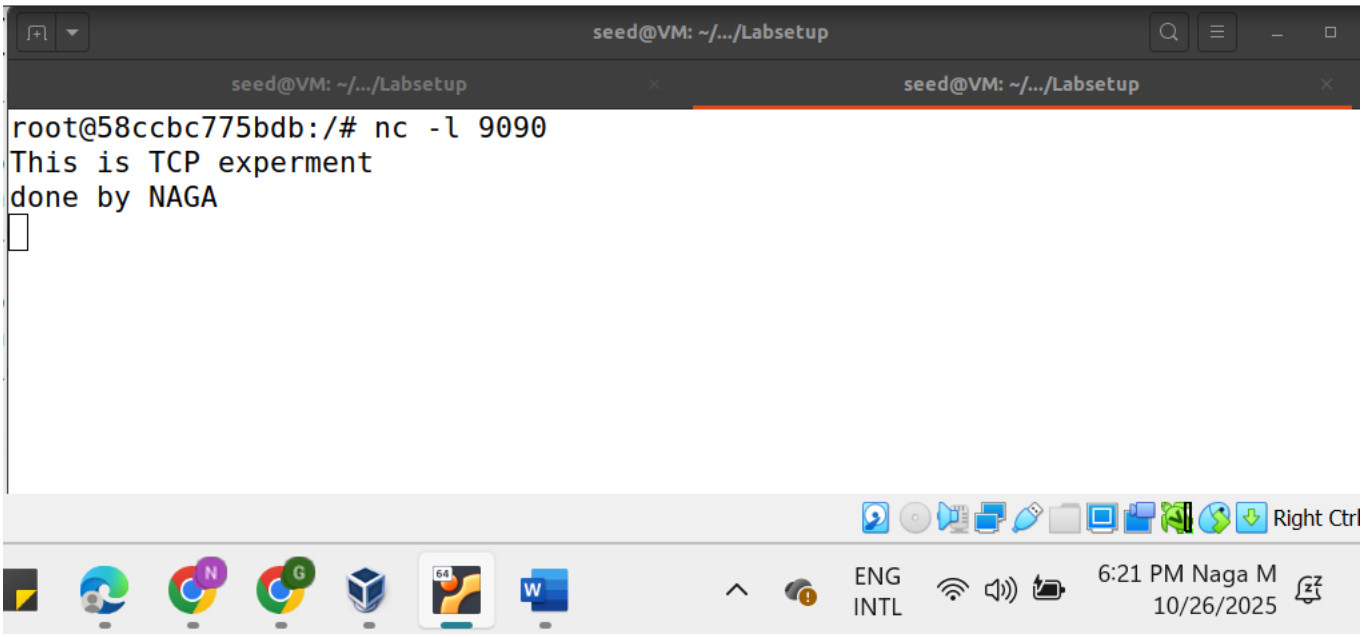


Based on your conntrack -L output, the UDP connection state will be kept for **15 seconds**.

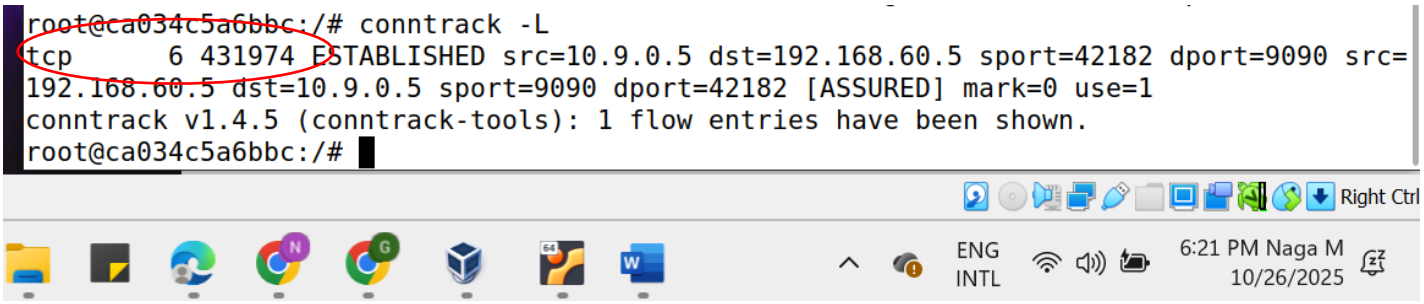
My observation from the result is that it was unidirectional UDP connection. Since our entry shows [UNREPLIED], this 15-second timeout applies specifically to unidirectional UDP traffic where no response was received.

TCPexperiment:





In the router container



Based on our conntrack -L output, the ESTABLISHED TCP connection state will be kept for **4,31,974 seconds.**

My observations: The connection shows ESTABLISHED state, meaning a successful TCP 3-way handshake occurred. The [ASSURED] flag confirms this is a confirmed two-way connection that won't be dropped early. This is the default timeout for established TCP connections in the Linux kernel

Task 3.B: Setting Up a Stateful Firewal

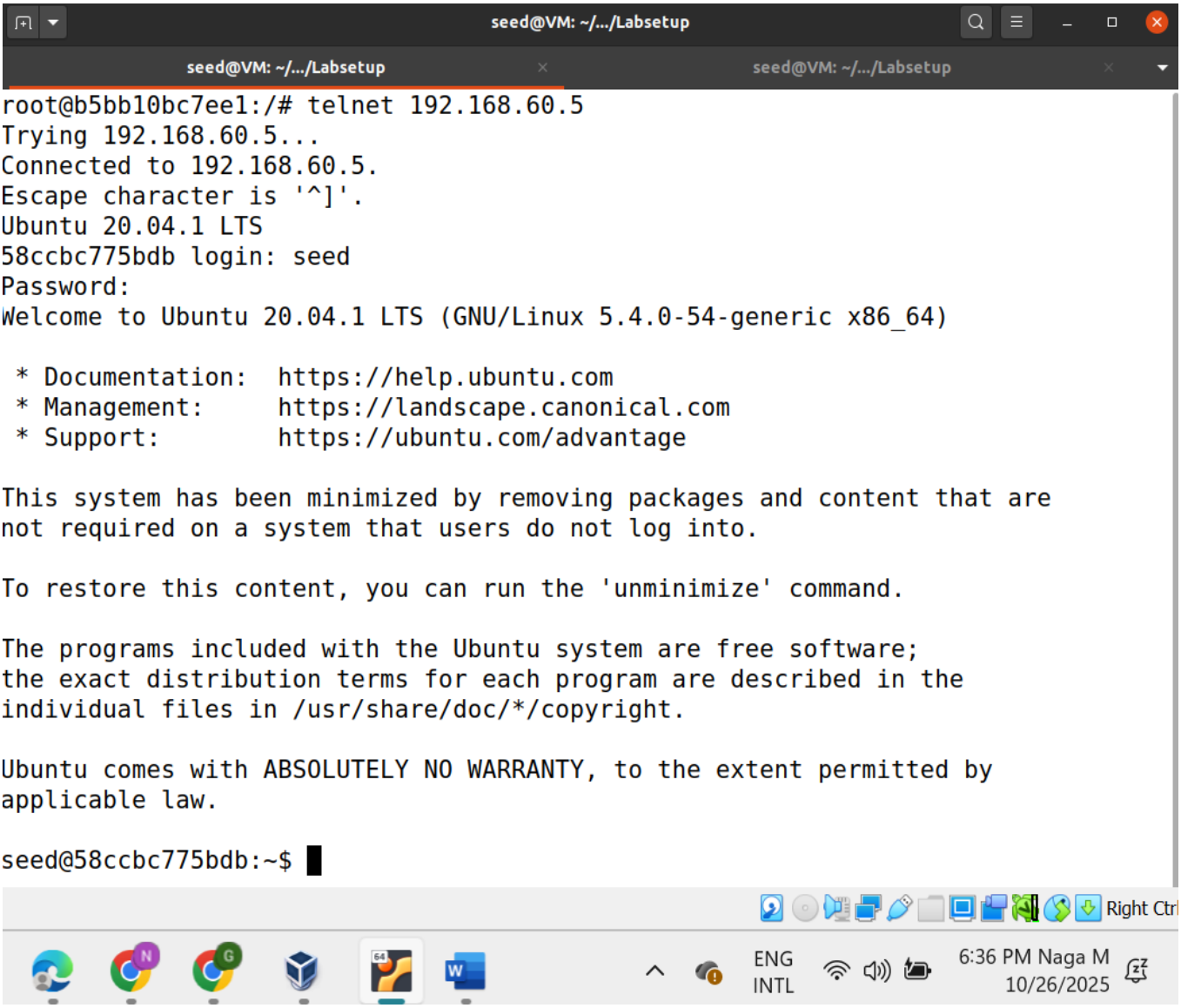
In this Task, we can Rewrite the rules from Task 2.C using the connection tracking mechanism. but this time, we will add a rule allowing internal hosts to visit any external server (this was not allowed in Task 2.C).

```
root@ca034c5a6bbc:/# iptables -P FORWARD ACCEPT
root@ca034c5a6bbc:/# iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
root@ca034c5a6bbc:/#
root@ca034c5a6bbc:/# iptables -A FORWARD -i eth0 -s 192.168.60.0/24 -m conntrack --ctstate NEW -j ACCEPT
root@ca034c5a6bbc:/# iptables -A FORWARD -i eth1 -d 192.168.60.5 -p tcp --dport 23 -m conntrack --ctstate NEW -j ACCEPT
root@ca034c5a6bbc:/# iptables -P FORWARD DROP
```

From an internal host (e.g., 192.168.60.6), try to ping an external host (10.9.0.5).

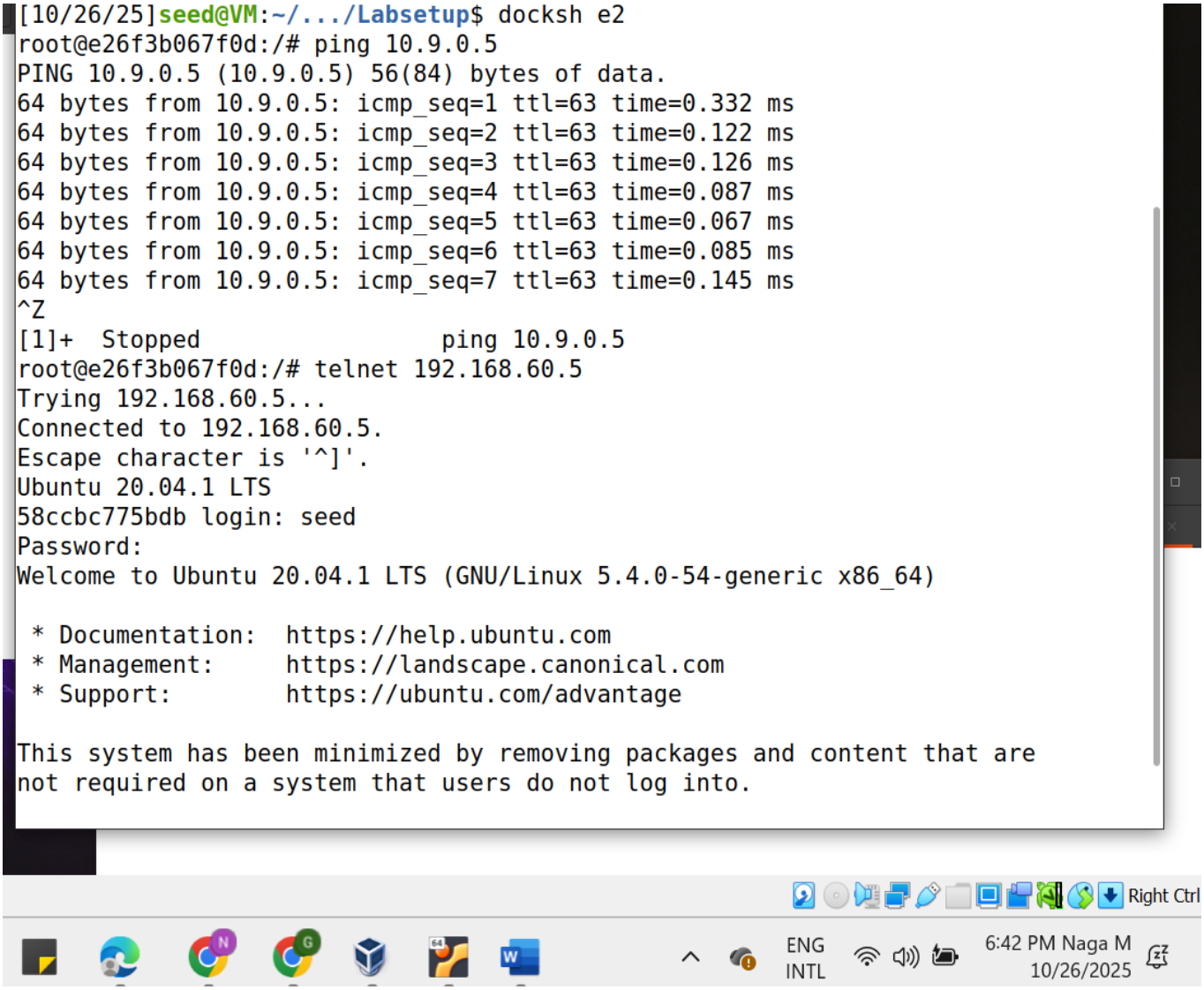
```
[10/26/25]seed@VM:~/.../Labsetup$ docksh e2
root@e26f3b067f0d:/# ping 10.9.0.5
PING 10.9.0.5 (10.9.0.5) 56(84) bytes of data.
64 bytes from 10.9.0.5: icmp_seq=1 ttl=63 time=0.332 ms
64 bytes from 10.9.0.5: icmp_seq=2 ttl=63 time=0.122 ms
64 bytes from 10.9.0.5: icmp_seq=3 ttl=63 time=0.126 ms
64 bytes from 10.9.0.5: icmp_seq=4 ttl=63 time=0.087 ms
64 bytes from 10.9.0.5: icmp_seq=5 ttl=63 time=0.067 ms
64 bytes from 10.9.0.5: icmp_seq=6 ttl=63 time=0.085 ms
64 bytes from 10.9.0.5: icmp_seq=7 ttl=63 time=0.145 ms
^Z
```

From Host A (10.9.0.5), try to telnet to the allowed server (192.168.60.5). It was worked.



Connnection established between internal networks

Try to telnet 192.168.60.5 from the 192.168.60.6..the connection was successful



Comparison: Stateful vs. Stateless

Stateful Firewall (This Task):

The **stateful approach is overwhelmingly superior** for modern network security because it aligns with how network protocols actually work. Rules are much simpler and more secure. A single rule allows internal hosts to access any external service without knowing the ports in advance. It understands the "context" of a conversation. Coming to **Disadvantage , it** Requires more memory and CPU to track the state of all connections.

Stateless Firewall (Task 2.C): No need to maintain connection state tables, making it very lightweight on memory and CPU. For basic port-based blocking, rules can be straightforward. Not limited by connection tracking table size. Coming to disadvantage Rules are complex and inflexible. To allow internal hosts to browse the web, we need multiple complex rules.

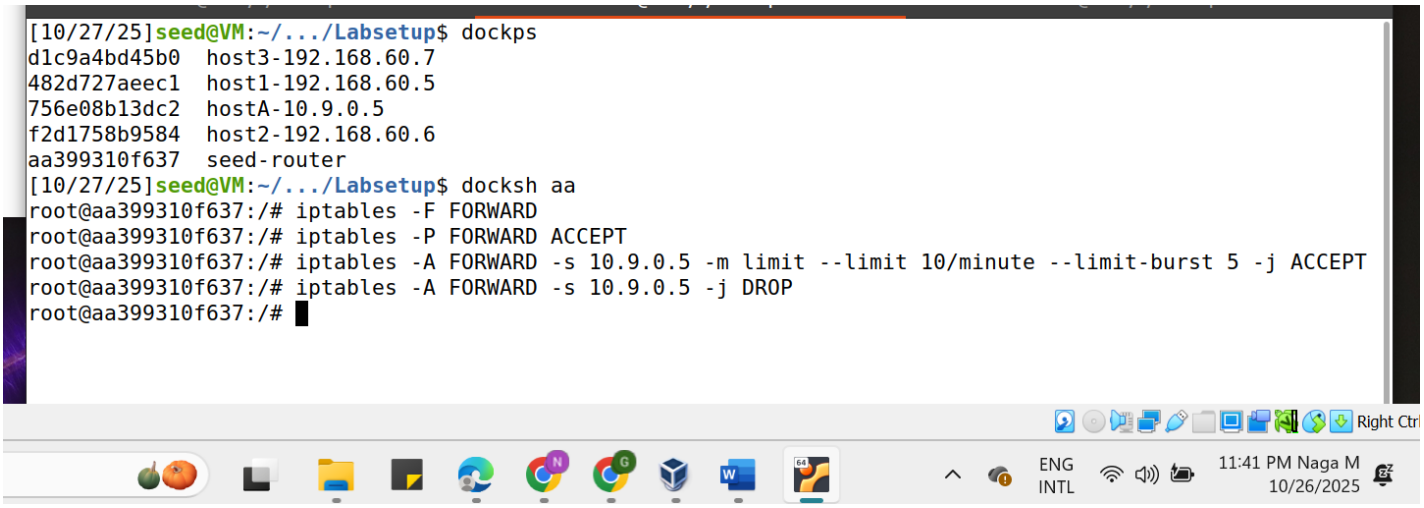
Task4: Limiting Network Traffic

To use the limit module in iptables to control the rate of packets passing through the firewall. We will limit how many ping (ICMP) packets from the external host 10.9.0.5 can reach the internal network.

Implement the Rate Limiting Rules : We will test two configurations: WITH and WITHOUT the second DROP rule.

Configuration A: WITH Both Rules (Recommended Setup)

Open the Router container, and run the following commands



and then ping 192.168.60.5 from 10.9.0.5.

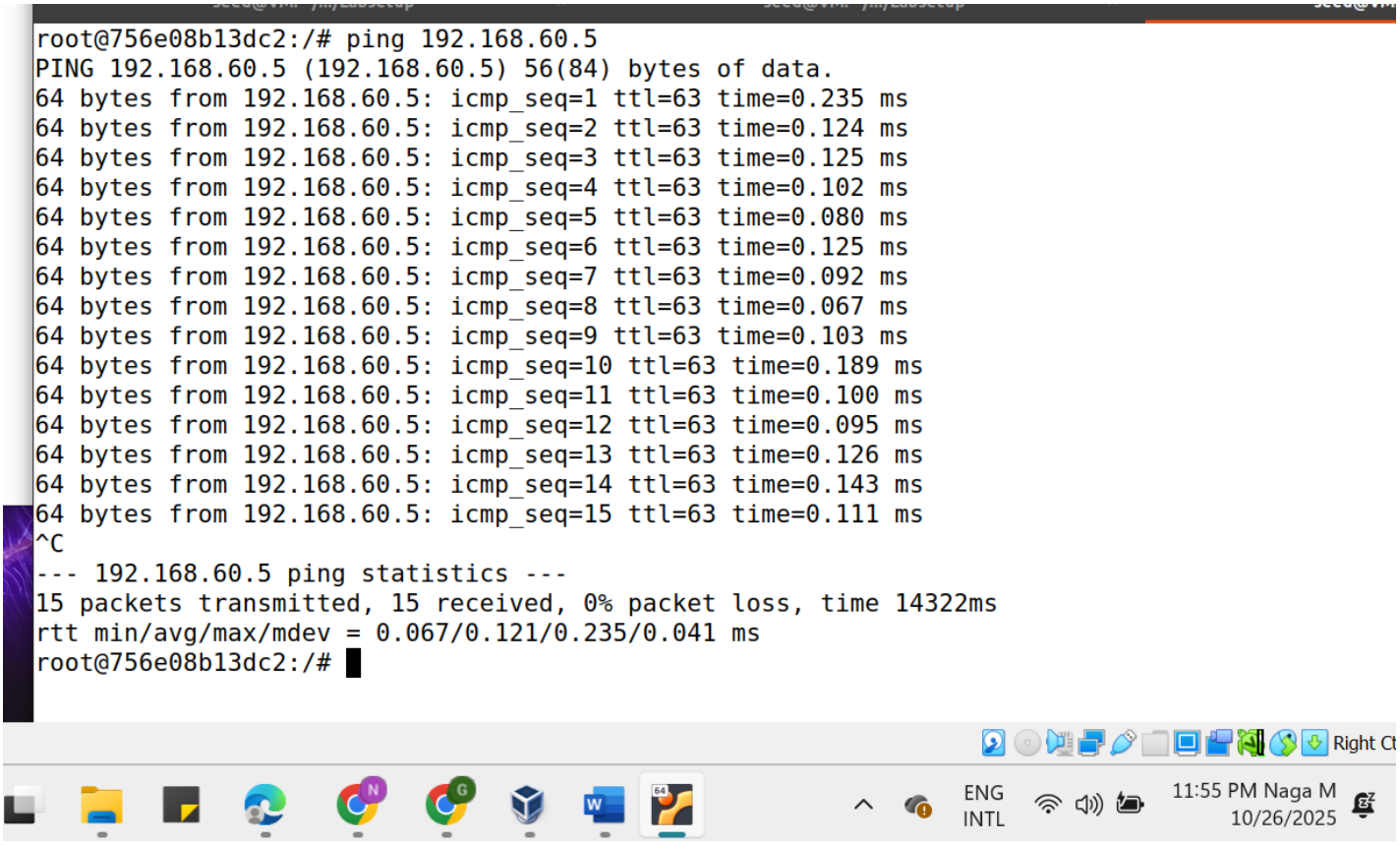
```
root@756e08b13dc2:/# ping 192.168.60.5
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data.
64 bytes from 192.168.60.5: icmp_seq=1 ttl=63 time=0.145 ms
64 bytes from 192.168.60.5: icmp_seq=2 ttl=63 time=0.153 ms
64 bytes from 192.168.60.5: icmp_seq=3 ttl=63 time=0.124 ms
64 bytes from 192.168.60.5: icmp_seq=4 ttl=63 time=0.125 ms
64 bytes from 192.168.60.5: icmp_seq=5 ttl=63 time=0.067 ms
64 bytes from 192.168.60.5: icmp_seq=7 ttl=63 time=0.064 ms
64 bytes from 192.168.60.5: icmp_seq=13 ttl=63 time=0.060 ms
64 bytes from 192.168.60.5: icmp_seq=19 ttl=63 time=0.120 ms
64 bytes from 192.168.60.5: icmp_seq=25 ttl=63 time=0.134 ms
64 bytes from 192.168.60.5: icmp_seq=31 ttl=63 time=0.139 ms
64 bytes from 192.168.60.5: icmp_seq=37 ttl=63 time=0.136 ms
64 bytes from 192.168.60.5: icmp_seq=42 ttl=63 time=0.132 ms
64 bytes from 192.168.60.5: icmp_seq=48 ttl=63 time=0.197 ms
64 bytes from 192.168.60.5: icmp_seq=54 ttl=63 time=0.098 ms
64 bytes from 192.168.60.5: icmp_seq=60 ttl=63 time=0.083 ms
64 bytes from 192.168.60.5: icmp_seq=66 ttl=63 time=0.123 ms
64 bytes from 192.168.60.5: icmp_seq=72 ttl=63 time=0.128 ms
64 bytes from 192.168.60.5: icmp_seq=78 ttl=63 time=0.127 ms
64 bytes from 192.168.60.5: icmp_seq=83 ttl=63 time=0.109 ms
64 bytes from 192.168.60.5: icmp_seq=89 ttl=63 time=0.222 ms
^X64 bytes from 192.168.60.5: icmp_seq=95 ttl=63 time=0.116 ms
^C
--- 192.168.60.5 ping statistics ---
95 packets transmitted, 21 received, 77.8947% packet loss, time 96373ms
rtt min/avg/max/mdev = 0.060/0.123/0.222/0.038 ms
root@756e08b13dc2:/#
```

This output clearly demonstrates that **Configuration A (with both rules)** is working exactly as expected.

From this output we can see that All first 5 packets succeeded immediately - this used our burst allowance. And then After burst exhaustion, we see the 10 packets/minute limit in action. This Pattern looks like Roughly 1 successful ping

every 6 seconds. our results show **real packet loss** - packets are actually being dropped when they exceed the rate limit. In this we can the packet loss also.

Without the second rule



During this experiment I observed that the First 5 pings were succeed immediately. After that they were limited to ~1 ping like every 6 seconds Even, Once the rate limit is exceeded, packets **WILL STILL BE ACCEPTED** by the default FORWARD policy. The rate limiting has **NO EFFECT** (**0% packet loss**)because packets that don't match the limit rule simply continue to the next rule (default ACCEPT)

Is the Second Rule Needed?

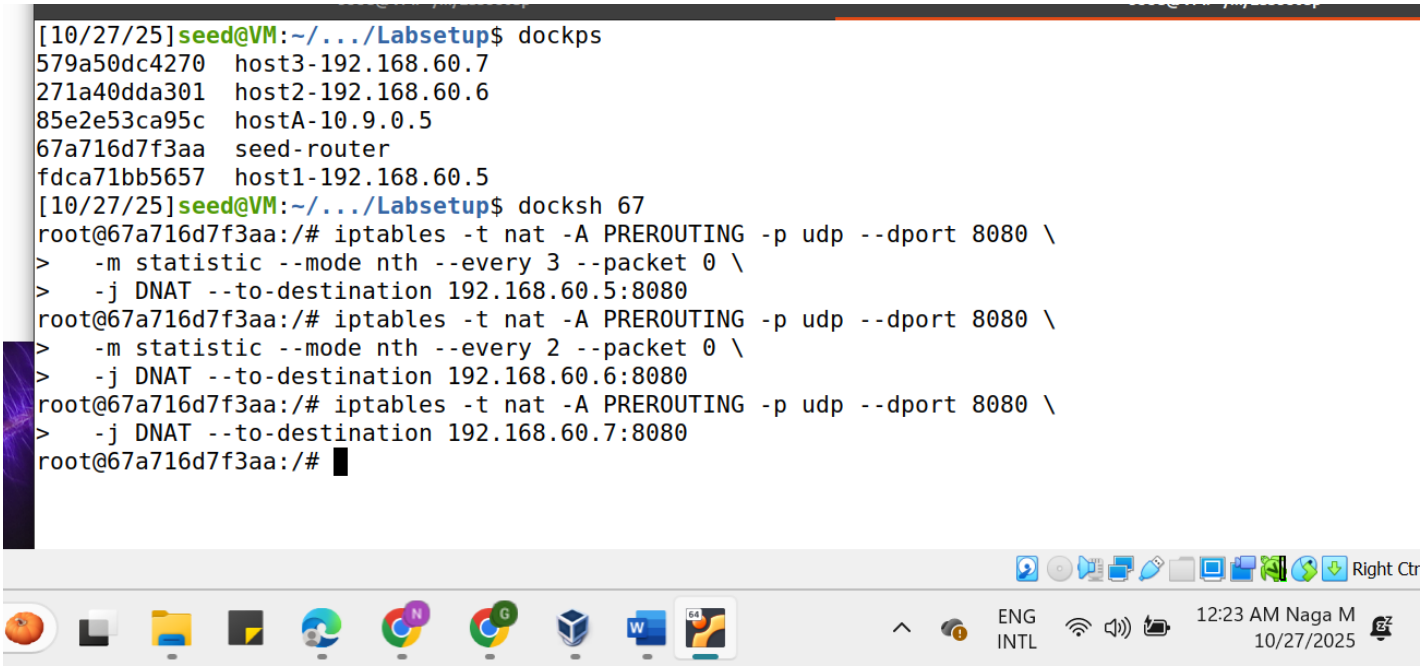
YES, ABSOLUTELY! our experiment results proves this conclusively. After seeing the both results we can come up with the second rule was essential. This is a critical lesson in iptables: Rules only affect packets that match them. Packets that don't match a rule continue to the next rule or the default policy.

Task5: LoadBalancing

Load Balancing involves using iptables on the **router container** to distribute incoming UDP traffic across three internal servers.

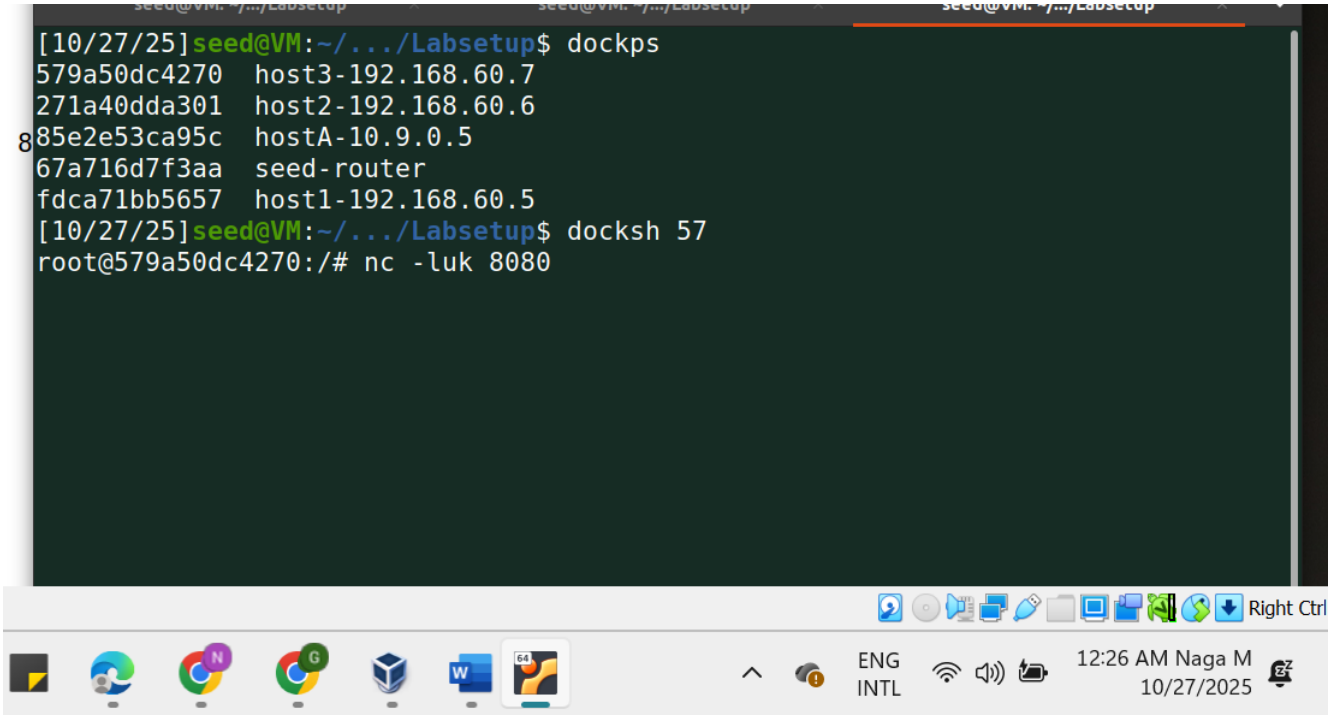
A. Using nth Mode (Round-Robin)

Distribute packets evenly in a round-robin fashion.



host1-192.168.60.5, host2-192.168.60.6, host3-192.168.60.7

On EACH of these three containers, run: nc -luk 8080

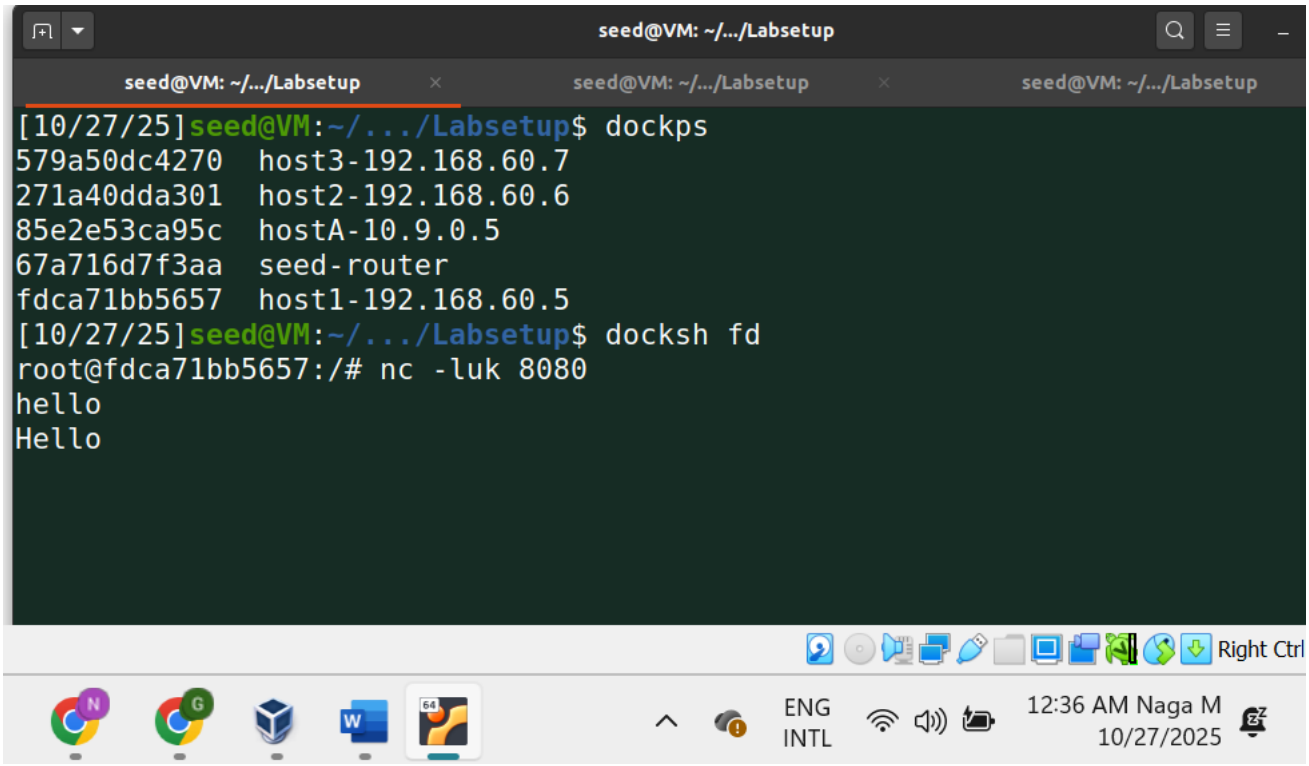


```
bash: nc-u: command not found
root@85e2e53ca95c:/#  echo hello | nc -u 10.9.0.11 8080
hi
naga
^C
root@85e2e53ca95c:/#  echo hello1 | nc -u 10.9.0.11 8080
^C
root@85e2e53ca95c:/#  echo hello_host2 | nc -u 10.9.0.11 8080
^C
root@85e2e53ca95c:/#  echo Hello | nc -u 10.9.0.11 8080
^C
root@85e2e53ca95c:/#  echo This is naga | nc -u 10.9.0.11 8080
^C
root@85e2e53ca95c:/#
root@85e2e53ca95c:/#  echo cyber student | nc -u 10.9.0.11 8080
^C
```

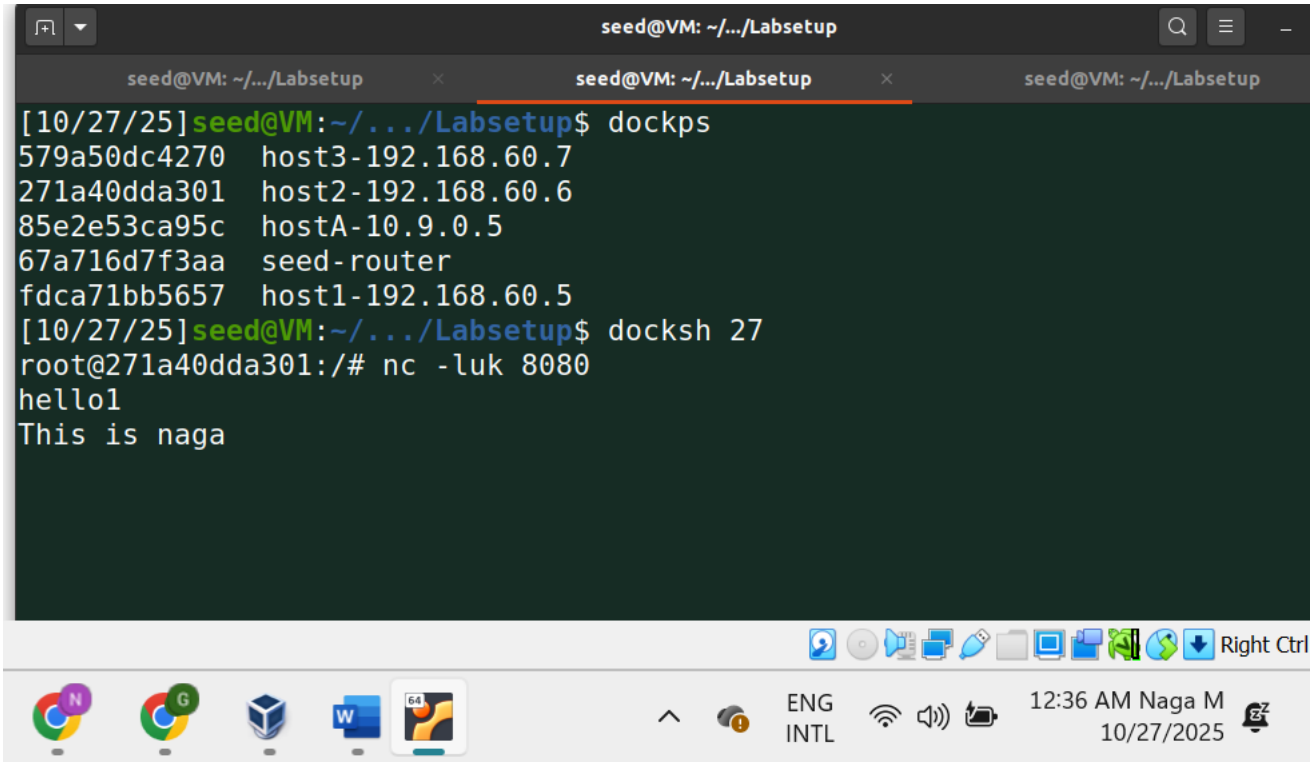
Right Ctrl

ENG INTL 12:35 AM Naga M 10/27/2025

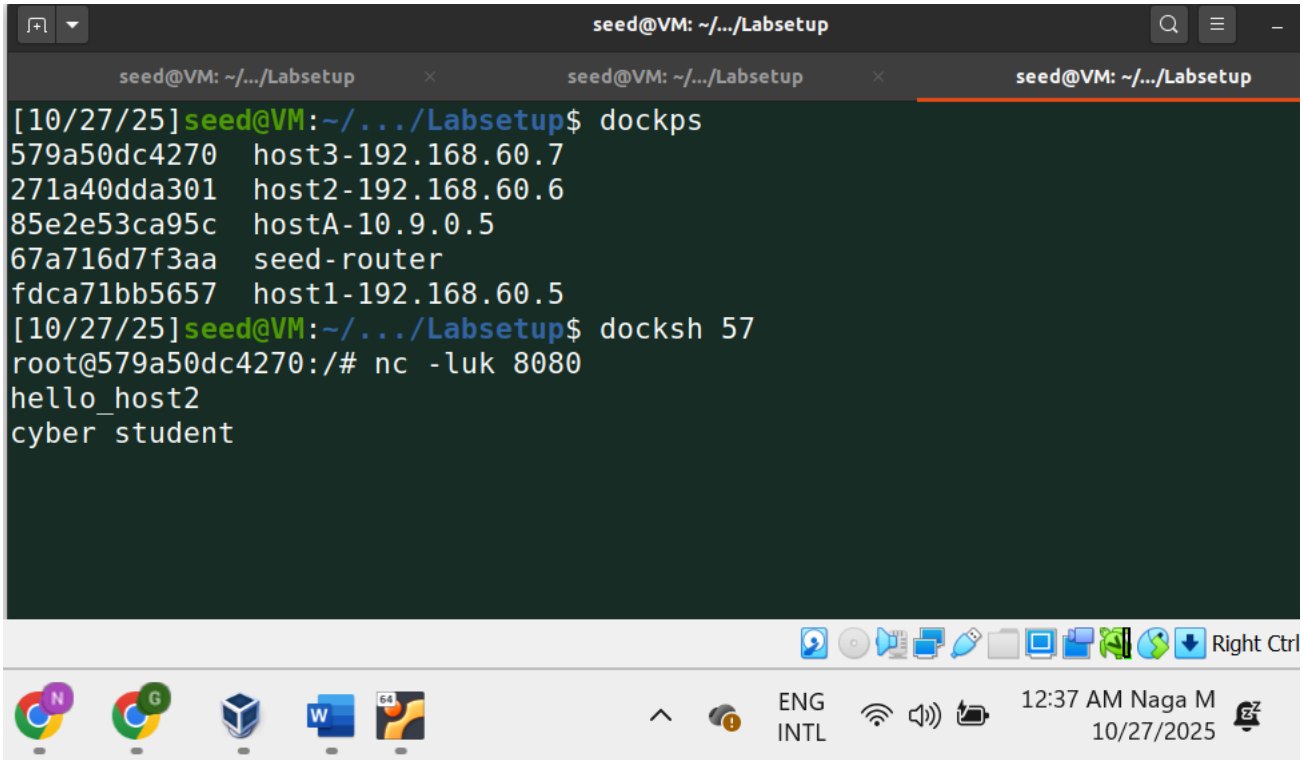
Lets see the Host1 container



And then Host2



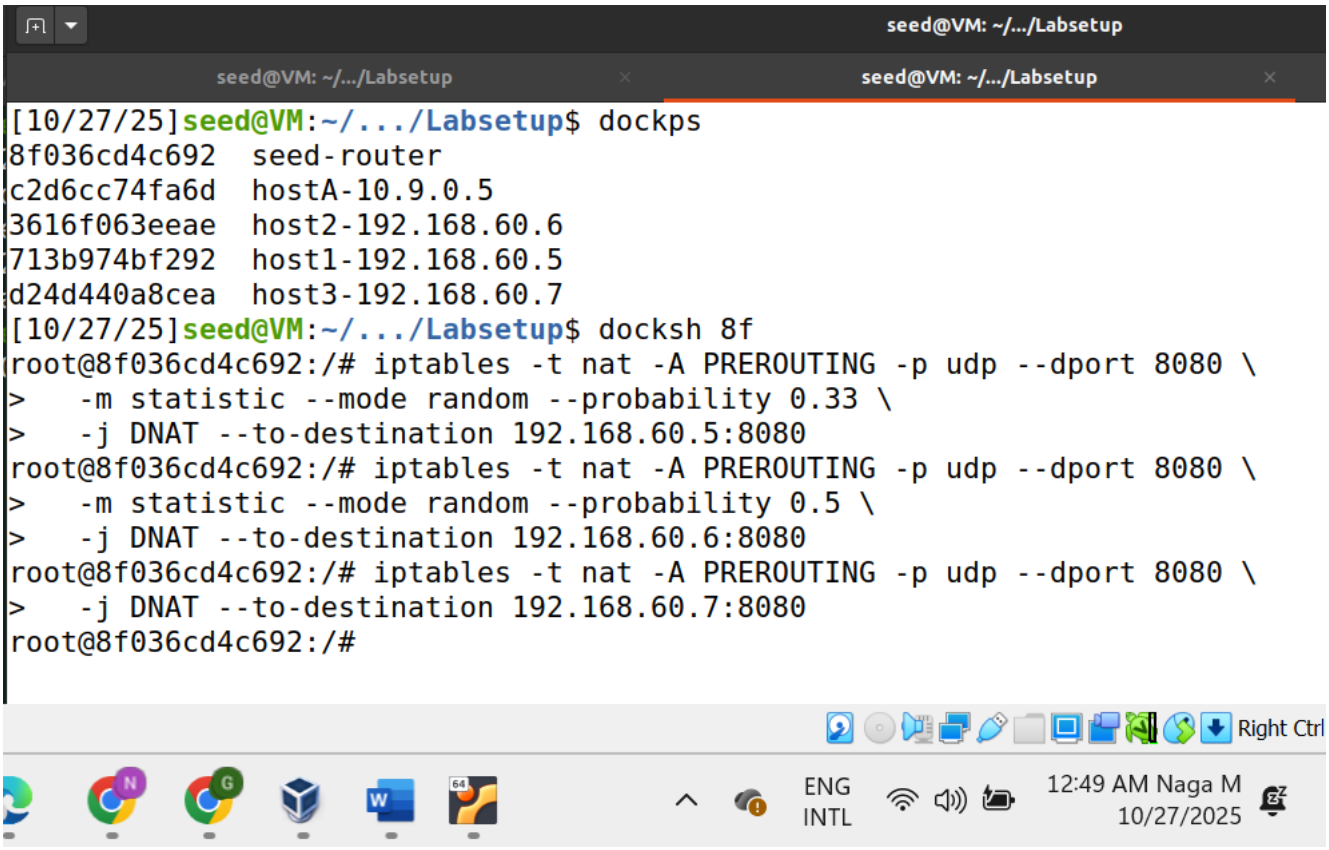
Now Host3



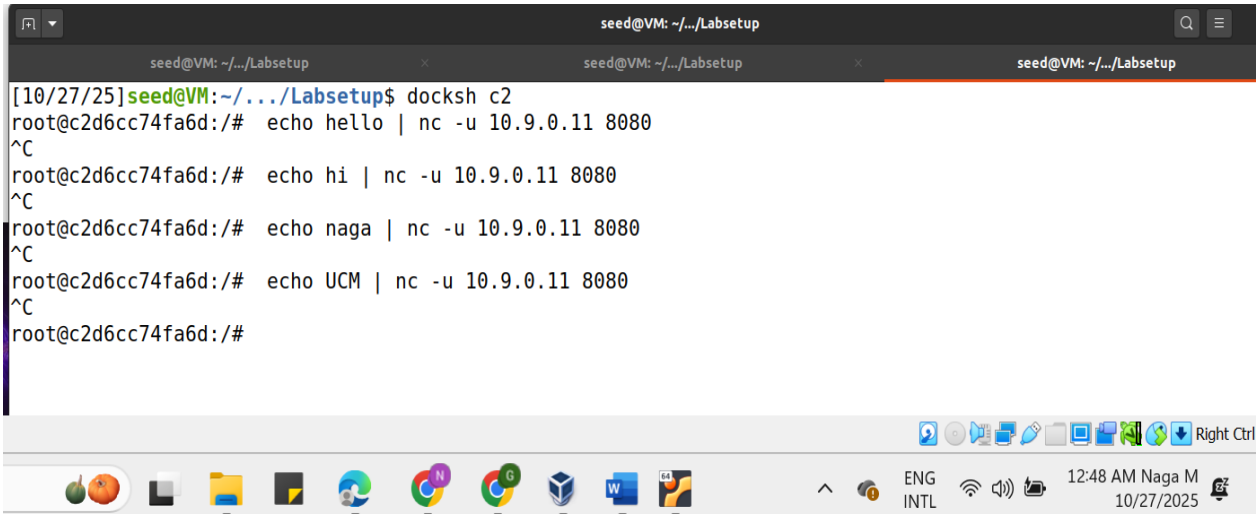
We can that evenly that the messages are evenly distributed from host1 to host3. From this output we cann see that the **1st packet** of every 3 sendto 192.168.60.5. and then the **1st packet** of the remaining 2 to 192.168.60.6 Finally, it sends all remaining packets to 192.168.60.7. so all the three internal hosts get the equal number of packets.

Using the random mode.

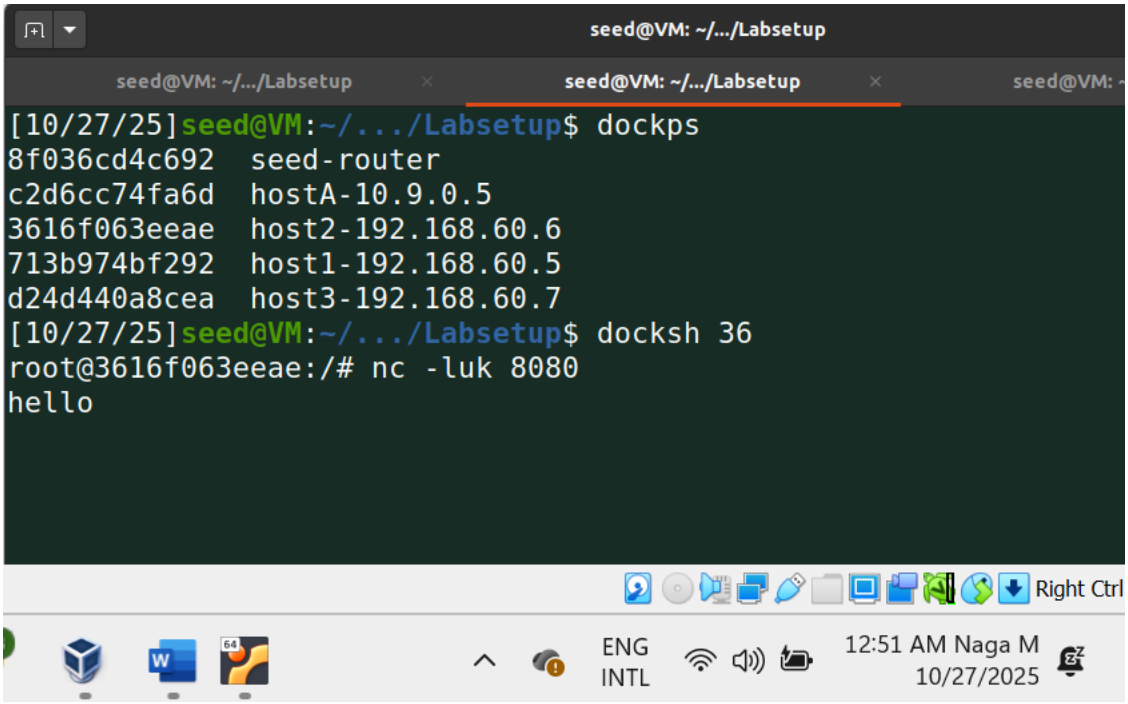
Before moving to this task lets restart the contains and then Repeat the same for the host1, host2, host3. And then open the router container and run the following commands



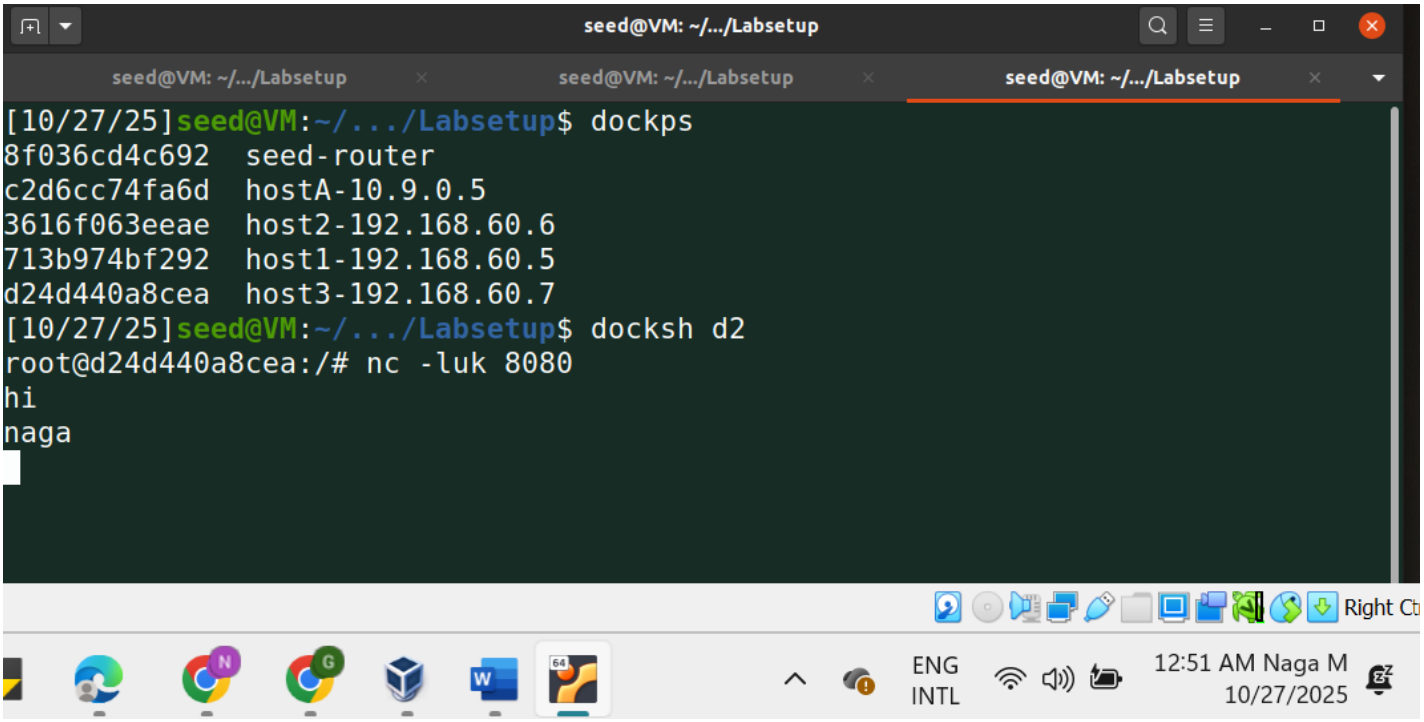
Now open the attacker container(10.9.0.5) send the messeges…



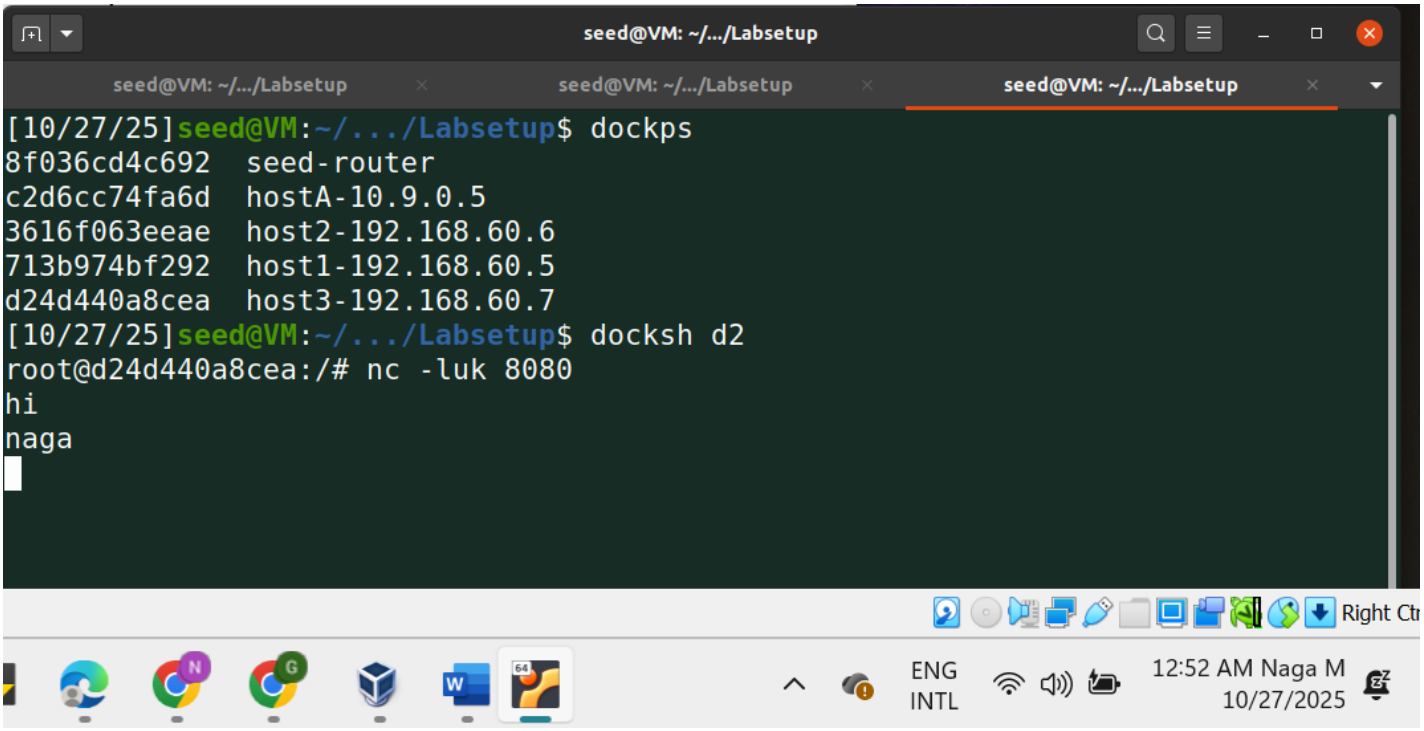
First I send the hello me, it was appeared on



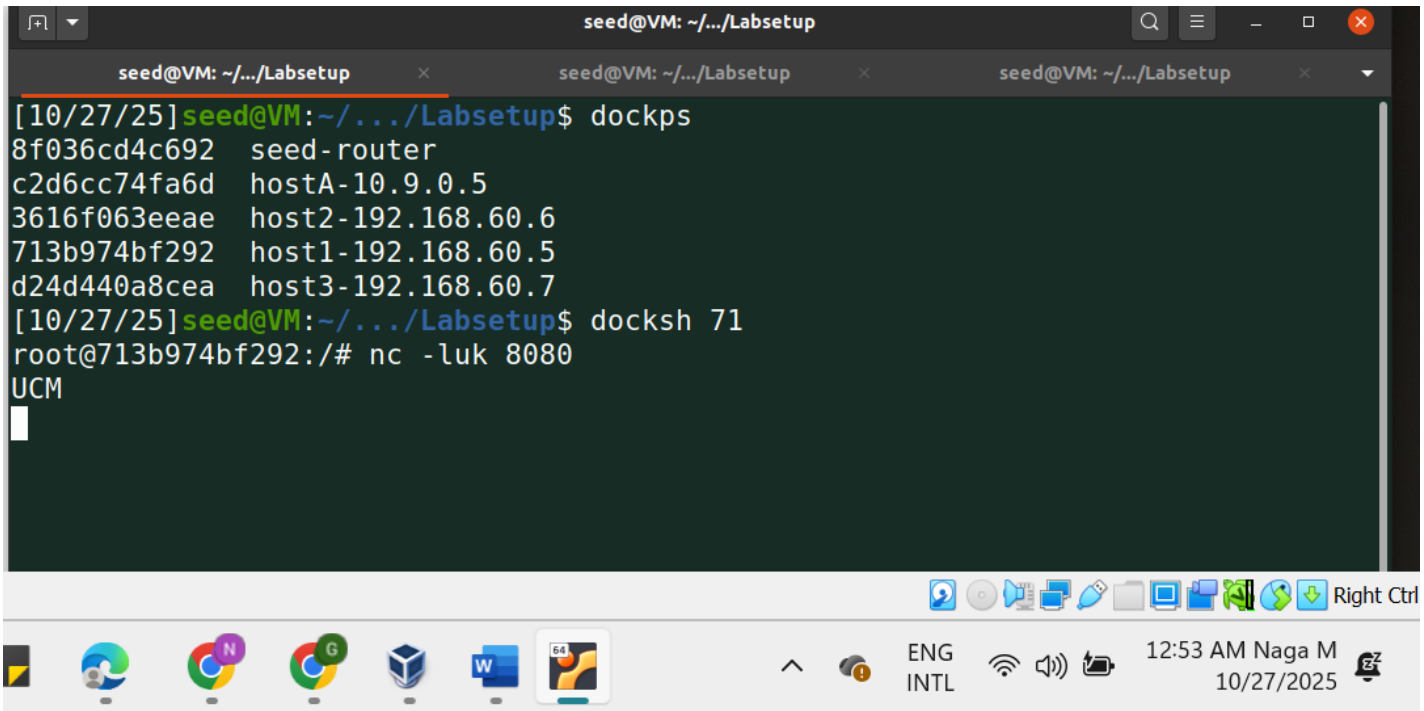
Then I send the hi message it was appeared in host3



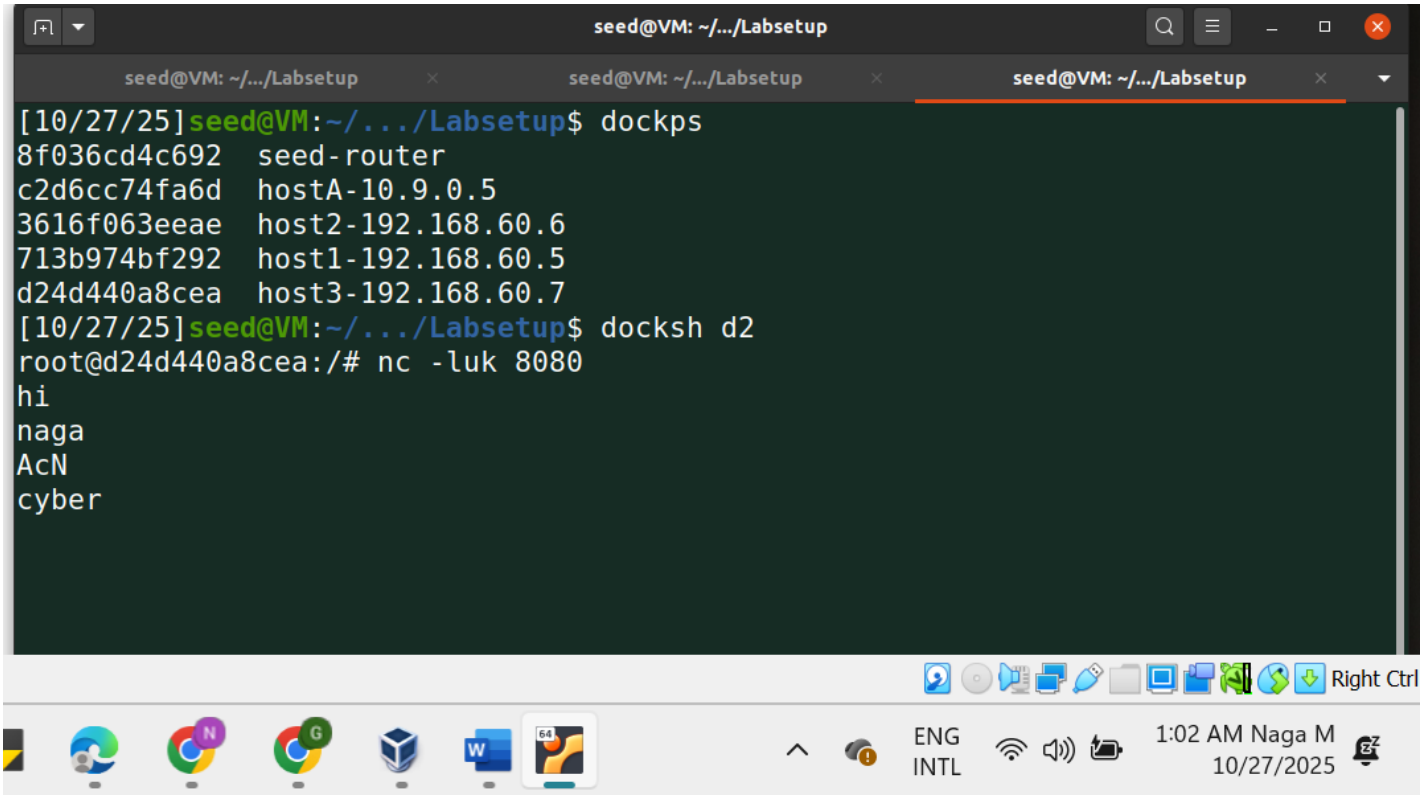
Then I send naga, this was appeared in the same host-3,



Last I send the UCM, this was appeared in host1



Then I gave CAN, cyber messages those are appeared in host3



From these outputs we can see that , host-3 get the more messages remaining two hosts get the equivalent messages. This setup distributes UDP packets randomly with equal probability across three servers. Each rule handles approximately one-third of the traffic, providing statistical load balancing that evens out over many packets.